Vietnam National University, Ho Chi Minh City
University of Technology
Faculty of Computer Science and Engineering



Microprocessors–Microcontrollers (Lab) (CO3010)

Laboratory Report

LAB_03: BUTTONS/SWITCHES TRAFFIC LIGHT APPLICATION

Instructor(s): Phan Văn Sỹ, CSE-HCMUT
Class: CC03
Student: Vo Le Thanh Trieu – 2453297

Ho Chi Minh City, October 2026

Contents

1 Buttons/Switches – Traffic Light Application **2**
1.1 Exercise 1: Sketch an FSM . . . 2
1.1.1 Requirement . . . 2
1.1.2 Overview of the proposed solution . . . 2
1.1.3 FSM A: Button processing . . . 3
1.1.4 FSM B: Mode selection . . . 4
1.1.5 FSM C: Traffic light . . . 6
1.1.6 Interaction between the three FSMs . . . 7
1.2 Exercise 2: Proteus Schematic . . . 8
1.2.1 Requirement . . . 8
1.2.2 Schematic . . . 8
1.2.3 GPIO configuration . . . 9
1.2.4 Circuit description . . . 9
1.3 Exercise 3: Create STM32 Project . . . 10
1.3.1 Requirement . . . 10
1.3.2 Project configuration . . . 10
1.3.3 GPIO configuration . . . 11
1.3.4 Timer configuration . . . 12
1.4 Exercise 4: Modify Timer Parameters . . . 13
1.4.1 Requirement . . . 13
1.4.2 Implementation idea . . . 13
1.4.3 Software-timer module . . . 14
1.4.4 Usage in main.c . . . 15
1.4.5 Changing the interrupt period . . . 16
1.4.6 Result . . . 16
1.5 Exercise 5: Adding Code for Button Debouncing . . . 16
1.5.1 Requirement . . . 16
1.5.2 Implementation . . . 16
1.5.3 Result . . . 24
1.6 Exercise 6: Adding Code for Displaying Modes . . . 24
1.6.1 Requirement . . . 24
1.6.2 Implementation . . . 24
1.6.3 Result . . . 33
1.7 Exercise 7: Adding Code for Increasing Time Duration Value for the Red LEDs . . . 33
1.7.1 Requirement . . . 33
1.7.2 Implementation . . . 33
1.7.3 Result . . . 37
1.8 Exercise 8: Adding Code for Increasing Time Duration Value for the Amber LEDs 38
1.8.1 Requirement . . . 38
1.8.2 Implementation . . . 38
1.8.3 Result . . . 42
1.9 Exercise 9: Adding Code for Increasing Time Duration Value for the Green LEDs . 42
1.9.1 Requirement . . . 42
1.9.2 Implementation . . . 42
1.9.3 Result . . . 47
1.10 Exercise 10: To Finish the Project . . . 47
1.10.1 Requirement . . . 47
1.10.2 Implementation . . . 48
1.10.3 Result . . . 56

References **58**

1 Buttons/Switches – Traffic Light Application

This report follows Exercises 1–10 of *Lab 03: Buttons/Switches*.

STM32CubeMX configures the STM32F103C6; application code is edited and built in Visual Studio Code with CMake. Proteus provides the circuit simulation. As in the previous lab, the exercises build one application step by step: later exercises extend the earlier ones.

The application is a traffic light for a crossroad. According to the specification in the lab manual, it contains:

- 12 single LEDs: 4 red, 4 amber, and 4 green;
- 4 seven-segment LEDs, two for each road, showing the time of the current light;
- 3 buttons, used to select the mode, to increase a time duration, and to set the chosen value;
- at least 4 modes: Mode 1 is the normal mode, while Modes 2, 3, and 4 modify the durations of the red, amber, and green LEDs, respectively.

Table 1: Purpose of the three buttons

Button	Switch	GPIO pin	Function
B1	sw0	PA11	Select the mode: $1 \to 2 \to 3 \to 4 \to 1$
B2	sw1	PA12	Increase the edited duration (range 1–99)
B3	sw2	PA13	Set (save) the edited duration

1.1 Exercise 1: Sketch an FSM

1.1.1 Requirement

Your task in this exercise is to sketch an FSM that describes your idea of how to solve the problem.

1.1.2 Overview of the proposed solution

The problem is divided into three small finite state machines (FSMs). Each FSM has one responsibility, and all three are executed side by side inside the `while (1)` loop. None of them waits or blocks; each call checks its inputs once, possibly changes state, and returns.

Table 2: The three FSMs of the proposed solution

FSM	Input	Output
A. Button FSM (one copy per button)	Debounced button level	Clean events: one press, or a press held longer than 1 s
B. Mode FSM	Events from B1, B2, and B3	Current mode, edited value, saved durations
C. Traffic-light FSM	1-second ticks and the saved durations	LED colours and countdown values

The three FSMs form layers. The button FSM converts a noisy mechanical signal into reliable events, the mode FSM decides what each event means, and the traffic-light FSM runs the crossroad only while the system is in Mode 1.

1.1.2.1 Notation used in the diagrams.

- An **ellipse** is a state. The FSM is always in exactly one state.
- The **filled dot** marks the initial state.
- An **arrow** between two states is a transition. A **self-loop** is a transition that returns to the same state.
- A label has the form `condition / action`. The part before `/` is the condition for taking the transition; the part after `/` is the action performed. A label without `/` only changes the state.
- The operators follow C syntax: `!` means NOT, and `&&` means AND.

1.1.3 FSM A: Button processing

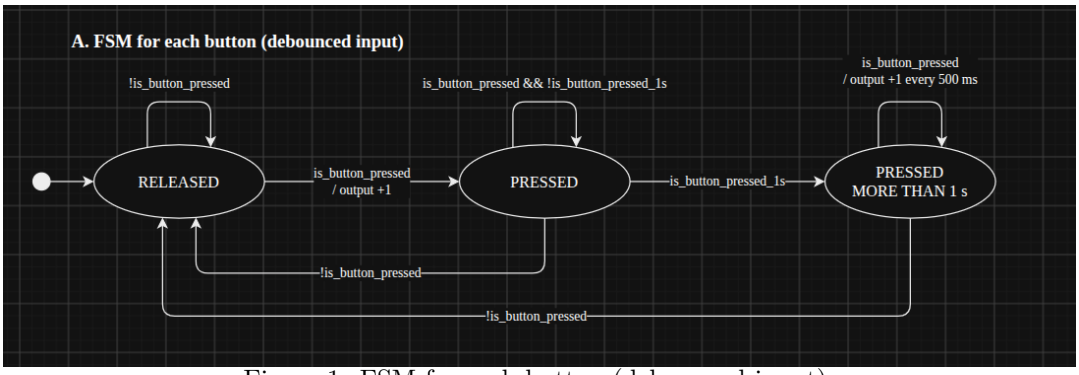


Figure 1: FSM for each button (debounced input).

Every mechanical button bounces for up to about 10 ms when it is pressed or released. Without processing, a single press could be read as several presses: one press of B1 could jump from Mode 1 to Mode 4, and one press of B2 could increase the value by more than one. The input is therefore debounced first, as in Program 1.2 of the lab manual: the pin is read every 10 ms, and a new level is accepted only when two consecutive readings are equal.

The FSM in Figure 1 uses the debounced level to produce events. It is the same for all three buttons, so one generic implementation is reused with the button index 0, 1, or 2.

The FSM uses two input functions:

- `is_button_pressed` is true while the debounced button is pressed;
- `is_button_pressed_1s` is true when the button has been held continuously for more than one second.

Table 3: States of the button FSM

State	Meaning
RELEASED	The button is not pressed. This is the initial state.
PRESSED	The button has just been pressed and has been held for less than one second.
PRESSED MORE THAN 1 s	The button is being held for a long time; the value increases automatically.

Table 4: Transitions of the button FSM

From → To	Label	Meaning
RELEASED → RELEASED	`!is_button_pressed`	No press: stay.
RELEASED → PRESSED	`is_button_pressed / output +1`	New press: output +1 exactly once.
PRESSED → PRESSED	`is_button_pressed && !is_button_pressed_1s`	Still held, less than 1 s: stay, no output.
PRESSED → PRESSED MORE THAN 1 s	`is_button_pressed_1s`	Held for 1 s: start auto-increase.
PRESSED MORE THAN 1 s → itself	`is_button_pressed / output +1 every 500 ms`	Still held: output +1 every 500 ms.
PRESSED → RELEASED	`!is_button_pressed`	Released before 1 s.
PRESSED MORE THAN 1 s → RELEASED	`!is_button_pressed`	Released after a long press.

The action `output +1` is performed only when the FSM *enters* PRESSED, not while it stays there. Hence one physical press produces exactly one event, however long the button is held. The third state is mainly useful for B2: a duration such as 50 can be reached by holding the button instead of pressing it 50 times. For B1 and B3, the mode FSM uses only the single-press event.

The FSM is complete and deterministic. In every state, the outgoing conditions cover all possible inputs, and exactly one of them is true. In PRESSED, `is_button_pressed_1s` can only be true while the button is pressed, so the three conditions never overlap.

1.1.4 FSM B: Mode selection

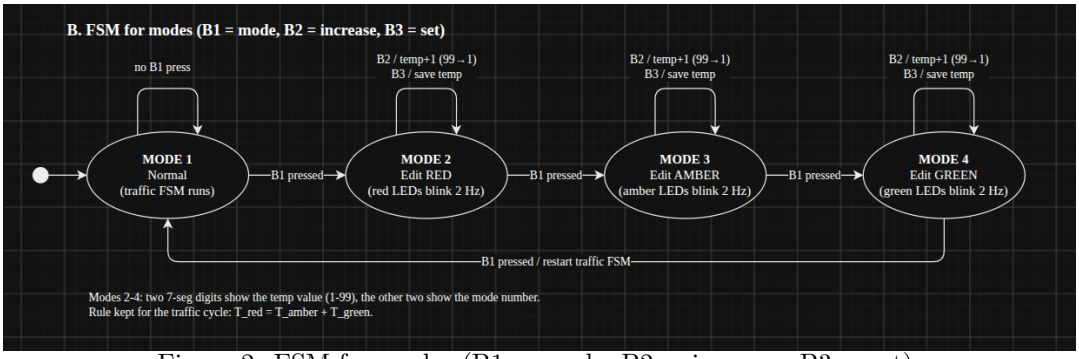


Figure 2: FSM for modes (B1 = mode, B2 = increase, B3 = set).

The mode FSM in Figure 2 implements the four modes of the specification. The label `B1 pressed` denotes one press event of B1 produced by FSM A.

Table 5: States of the mode FSM

State	Behaviour while in the state
MODE 1 – Normal	The traffic light runs normally; FSM C is active.
MODE 2 – Edit RED	All red LEDs blink at 2 Hz.
MODE 3 – Edit AMBER	All amber LEDs blink at 2 Hz.
MODE 4 – Edit GREEN	All green LEDs blink at 2 Hz.

In Modes 2–4, two seven-segment LEDs display the edited value, and the other two display the mode number. Blinking at 2 Hz means one complete on–off cycle every 500 ms, so the LEDs change state every 250 ms.

Table 6: Transitions of the mode FSM

From → To	Label	Meaning
MODE 1 → MODE 1	`no B1 press`	Stay; the traffic light keeps running.
MODE 1 → 2 → 3 → 4	`B1 pressed`	Each press of B1 selects the next mode.
MODE 4 → MODE 1	`B1 pressed / restart traffic FSM`	Return to normal operation; FSM C restarts from S0 with the saved durations.
MODE 2, 3, 4 → itself	`B2 / temp+1 (99→1)`	Increase the temporary value; after 99 it returns to 1.
MODE 2, 3, 4 → itself	`B3 / save temp`	Copy the temporary value into the duration of the edited colour.

The self-loop drawn above each edit mode combines two transitions, one for B2 and one for B3; both keep the FSM in the same mode.

B2 changes only a temporary value, `temp`, which acts as a draft. When an edit mode is entered, `temp` starts from the current duration of that colour. B3 copies `temp` into the real duration. If B1 is pressed without pressing B3, the draft is discarded and the duration remains unchanged. The range 1–99 matches the two digits available on each road.

Table 7 shows how the red duration is changed from 5 s to 8 s.

Table 7: Example: changing the red duration from 5 s to 8 s

Step	Action	Mode	temp	T_red	Displayed
0	None	1	–	5	Countdown
1	Press B1	2	5	5	05 and 02
2	Press B2 3 times	2	6, 7, 8	5	08 and 02
3	Press B3	2	8	8	08 and 02
4	Press B1 3 times	3, 4, 1	–	8	Red lasts 8 s

1.1.5 FSM C: Traffic light

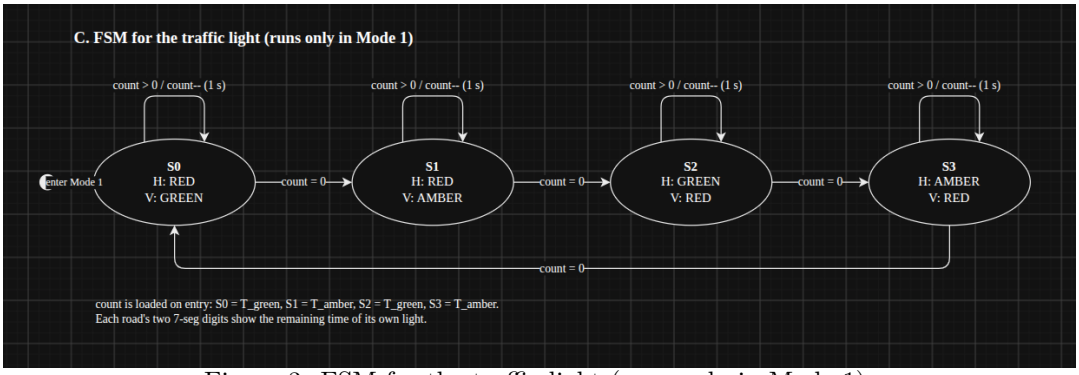


Figure 3: FSM for the traffic light (runs only in Mode 1).

The traffic-light FSM in Figure 3 runs only in Mode 1. H denotes the horizontal road and V the vertical road. Each time Mode 1 is entered, the FSM starts from S0.

Table 8: States of the traffic-light FSM

State	H road	V road	Duration	Description
S0	RED	GREEN	T_green	V traffic moves
S1	RED	AMBER	T_amber	V traffic prepares to stop
S2	GREEN	RED	T_green	H traffic moves
S3	AMBER	RED	T_amber	H traffic prepares to stop

When a state is entered, the counter `count` is loaded with the duration of that state. The self-loop `count > 0 / count-- (1 s)` keeps the FSM in the same state and decreases the counter once per second. The transition `count = 0` moves the FSM to the next state, and S3 returns to S0. Thus each road passes through green, amber, and red, and the two roads are never green at the same time.

The FSM uses only T_green and T_amber; there is no separate red state. A road is red while the other road is green and then amber:

$$T_{\text{red}} = T_{\text{green}} + T_{\text{amber}}.$$

This rule must remain true after the durations are edited. Otherwise, the red countdown shown on the seven-segment LEDs would not match the time for which the red light is actually on.

Each road's two seven-segment LEDs show the remaining time of its own light. Table 9 uses $T_{\text{red}} = 5$ s, $T_{\text{amber}} = 2$ s, and $T_{\text{green}} = 3$ s.

Table 9: Example traffic-light cycle

Time	State	H light	H display	V light	V display
0 s	S0	RED	05	GREEN	03
1 s	S0	RED	04	GREEN	02
2 s	S0	RED	03	GREEN	01
3 s	S1	RED	02	AMBER	02
4 s	S1	RED	01	AMBER	01
5 s	S2	GREEN	03	RED	05
6 s	S2	GREEN	02	RED	04
7 s	S2	GREEN	01	RED	03
8 s	S3	AMBER	02	RED	02
9 s	S3	AMBER	01	RED	01
10 s	S0	RED	05	GREEN	03

The red countdown of road H ends at the same moment as the amber light of road V, because $5 = 3 + 2$. One complete cycle lasts $2 \times (T_{\text{green}} + T_{\text{amber}}) = 10$ s.

1.1.6 Interaction between the three FSMs

- FSM A runs for each button and produces clean press events.
- FSM B consumes these events: B1 changes the mode, B2 increases `temp`, and B3 saves it.
- FSM C drives the LEDs and countdowns only in Mode 1. When B1 selects Mode 2, FSM C stops and the LEDs are used for blinking. When the system returns to Mode 1, FSM C restarts from S0.
- The 10-ms TIM2 interrupt provides the time base for debouncing, the 1-second countdown, and the 2-Hz blinking, without `HAL_Delay()`.

An FSM is implemented in C with one variable holding the current state and a `switch` statement called repeatedly from the main loop. Program 1.1.6 illustrates this for FSM A, following the structure of Program 1.7 in the lab manual.

Program 1.1: Mapping the button FSM to C

```
switch (buttonState) {
case BUTTON_RELEASED:
    if (is_button_pressed(i)) {
        buttonState = BUTTON_PRESSED;
        /* output +1 (exactly once) */
    }
    break;

case BUTTON_PRESSED:
    if (!is_button_pressed(i)) {
        buttonState = BUTTON_RELEASED;
    } else if (is_button_pressed_1s(i)) {
        buttonState = BUTTON_PRESSED_MORE_THAN_1_SECOND;
    }
    break;

case BUTTON_PRESSED_MORE_THAN_1_SECOND:
    if (!is_button_pressed(i)) {
        buttonState = BUTTON_RELEASED;
```

```
    }
    /* output +1 every 500 ms */
    break;
}
```

1.2 Exercise 2: Proteus Schematic

1.2.1 Requirement

Your task in this exercise is to draw a Proteus schematic for the problem above.

1.2.2 Schematic

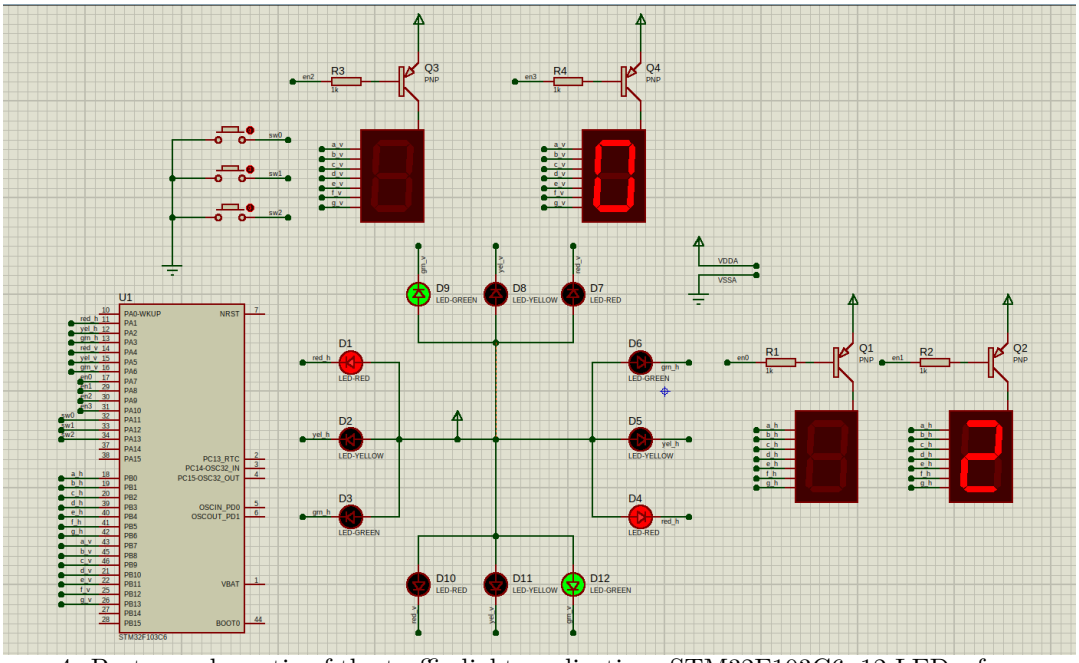


Figure 4: Proteus schematic of the traffic-light application: STM32F103C6, 12 LEDs, four seven-segment LEDs, and three buttons.

The circuit in Figure 4 contains every component required by the specification. All application code in Exercises 3–10 uses this schematic; the hardware does not change between exercises.

1.2.3 GPIO configuration

Table 10: GPIO assignments for Lab 03

GPIO pins	Signals	Function
PA1, PA2, PA3	red_h, yel_h, grn_h	Red, amber, and green LEDs of the horizontal road
PA4, PA5, PA6	red_v, yel_v, grn_v	Red, amber, and green LEDs of the vertical road
PA7, PA8	en0, en1	Enables of the two horizontal-road digits
PA9, PA10	en2, en3	Enables of the two vertical-road digits
PA11, PA12, PA13	sw0, sw1, sw2	Buttons B1 (mode), B2 (increase), B3 (set)
PB0–PB6	a_h–g_h	Segments of the horizontal-road digits
PB7–PB13	a_v–g_v	Segments of the vertical-road digits

PA0, PA14, PA15, PB14, and PB15 are not used. All LED, enable, and segment pins are push-pull outputs; the button pins are inputs.

1.2.4 Circuit description

1.2.4.1 Traffic LEDs. The crossroad has four directions, each with a red, an amber, and a green LED, giving 12 LEDs in total. The two directions of the same road always show the same colour, so each pair of LEDs shares one GPIO pin.

Table 11: LED pairs and their control signals

Road	Colour	LEDs	Signal
Horizontal	Red	D1, D4	red_h (PA1)
Horizontal	Amber	D2, D5	yel_h (PA2)
Horizontal	Green	D3, D6	grn_h (PA3)
Vertical	Red	D7, D10	red_v (PA4)
Vertical	Amber	D8, D11	yel_v (PA5)
Vertical	Green	D9, D12	grn_v (PA6)

Thus six GPIO pins control all 12 LEDs. The anodes of all LEDs are connected to the supply, and each cathode is connected to its GPIO pin. The LEDs are therefore active low: writing `GPIO_PIN_RESET` turns a pair on, and writing `GPIO_PIN_SET` turns it off.

1.2.4.2 Seven-segment LEDs. Each road has two common-anode seven-segment LEDs, showing a two-digit value from 00 to 99. As in Lab 02, the common anode of each digit is switched by a PNP transistor with a 1-kΩ base resistor:

- Q1 and Q2, controlled by en0 (PA7) and en1 (PA8), enable the two horizontal-road digits;
- Q3 and Q4, controlled by en2 (PA9) and en3 (PA10), enable the two vertical-road digits.

The segments and the enables are both active low. A digit is enabled by driving its enable pin low, and a segment is lit by driving its segment pin low.

Each road uses its own segment bus: PB0–PB6 for the horizontal road and PB7–PB13 for the vertical road. The two digits of a road share that road's bus and are scanned one at a time. Because the buses are independent, the first digit of both roads can be displayed at the same time,

followed by the second digit of both roads. One complete scan therefore needs only two steps instead of four, which keeps the display bright and steady.

1.2.4.3 Buttons. Each button connects its GPIO pin to ground when pressed. The pin must read high when the button is released, so a pull-up resistor is required, as explained in Section 3.1 of the lab manual. The internal pull-up resistors of PA11–PA13 are enabled in the STM32CubeMX configuration of Exercise 3. A button therefore reads `GPIO_PIN_SET` when released and `GPIO_PIN_RESET` when pressed.

1.2.4.4 Design notes.

- PA13 is also the SWDIO debug pin. To use it as a button input, the debug interface is disabled in STM32CubeMX (*System Core* → *SYS* → *Debug: No Debug*).
- Net labels in Proteus connect only when their names are identical; every segment label (for example, `a_h`) is therefore written in exactly the same way at the microcontroller and at the display.

1.3 Exercise 3: Create STM32 Project

1.3.1 Requirement

Your task in this exercise is to create a project that has pin corresponding to the Proteus schematic that you draw in previous section. You need to set up your timer interrupt is about 10ms.

1.3.2 Project configuration

The project is created in STM32CubeMX for the STM32F103C6Ux and generated with the CMake toolchain for Visual Studio Code. The CubeMX part (UFQFPN48) and the Proteus model (LQFP48) contain the same chip with the same pin assignment, so the firmware runs directly in Proteus. The project is configured in four steps:

1. **SYS:** Debug is set to *No Debug*. This releases PA13 (button B3) and PB3–PB4 (segments d_h and e_h), which are otherwise reserved for the SWD and JTAG debug interfaces.
2. **RCC and clock:** the internal 8-MHz HSI oscillator is the system clock, and the AHB, APB1, and APB2 prescalers are 1.
3. **GPIO:** the 27 pins and their user labels follow Table 10.
4. **TIM2:** configured for a 10-ms update interrupt, with the TIM2 global interrupt enabled in the NVIC.

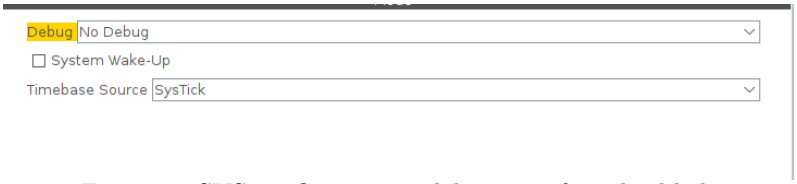


Figure 5: SYS configuration: debug interface disabled.

1.3.3 GPIO configuration

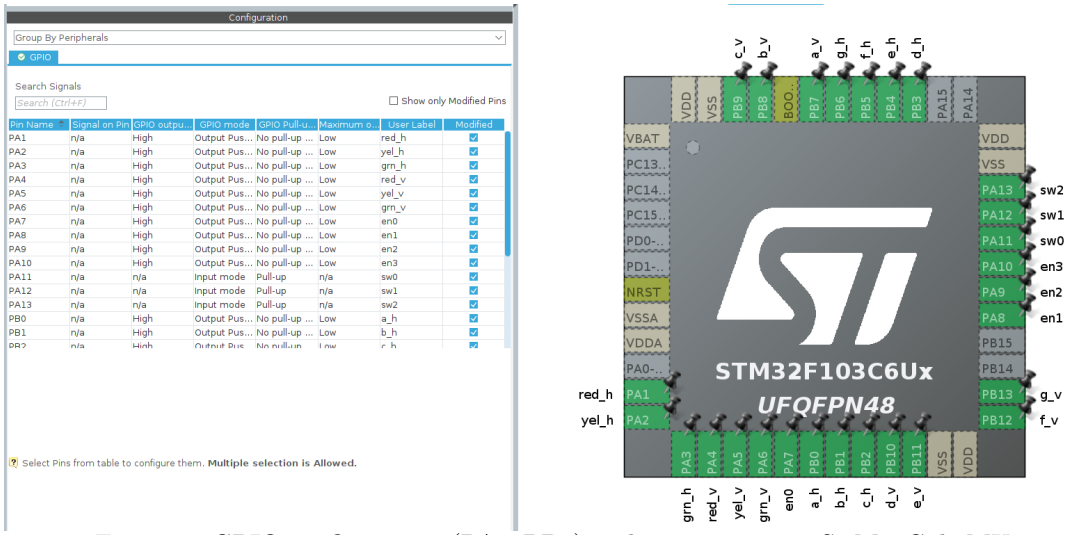


Figure 6: GPIO configuration (PA1–PB1) and pinout view in STM32CubeMX.

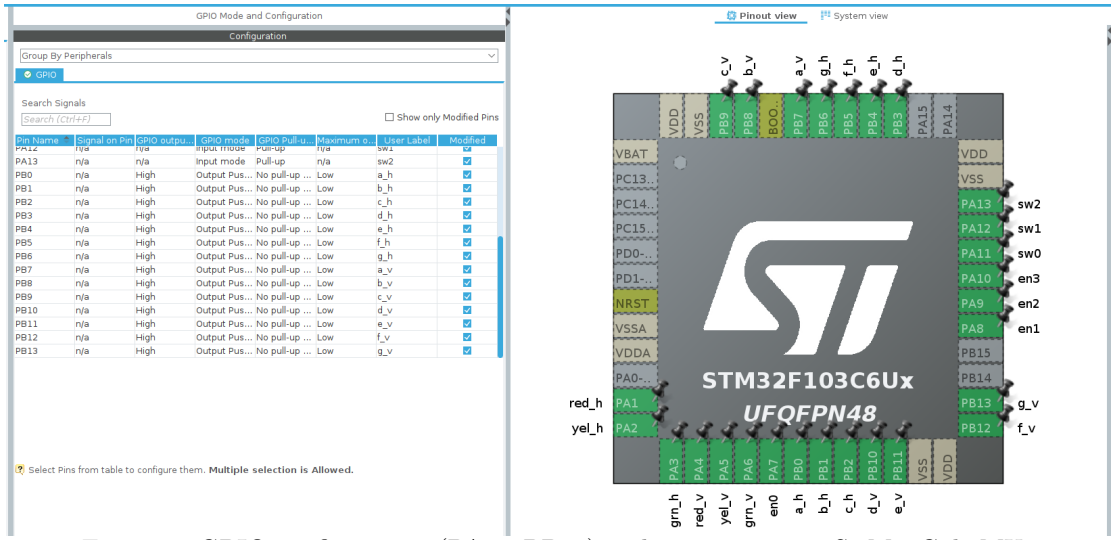


Figure 7: GPIO configuration (PA12–PB13) and pinout view in STM32CubeMX.

Table 12: GPIO settings in STM32CubeMX

Setting	**Outputs: PA1–PA10, PB0–PB13**	**Inputs: PA11–PA13**
Mode	Output push-pull	Input
Initial level	High	–
Pull resistor	None	Pull-up
Output speed	Low	–

Every pin and user label in Figures 6 and 7 matches the schematic in Figure 4. All outputs drive active-low circuits, so their initial level is High: every LED and digit is off after reset until the application turns it on. The button pins use the internal pull-up resistors, because the

schematic has no external ones; a released button reads `GPIO_PIN_SET`, and a pressed button reads `GPIO_PIN_RESET`. CubeMX generates a macro for every label in `main.h`, for example `red_h_Pin` and `red_h_GPIO_Port`, so the code uses the same names as the schematic.

1.3.4 Timer configuration

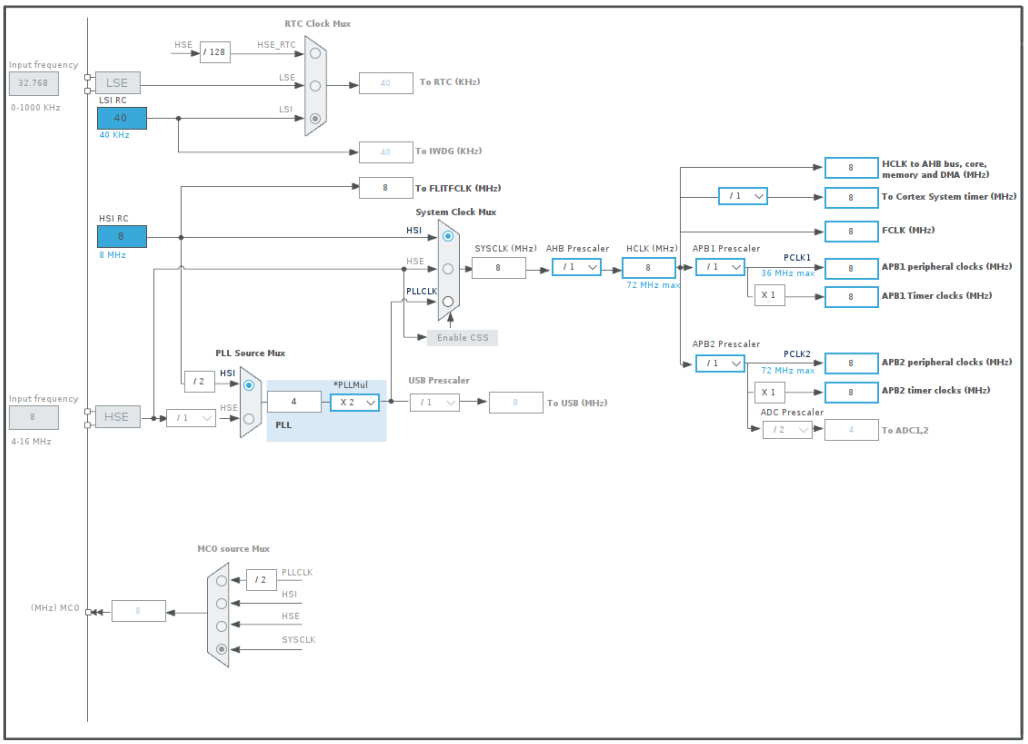


Figure 8: Clock configuration: 8-MHz HSI system and APB1 timer clocks.

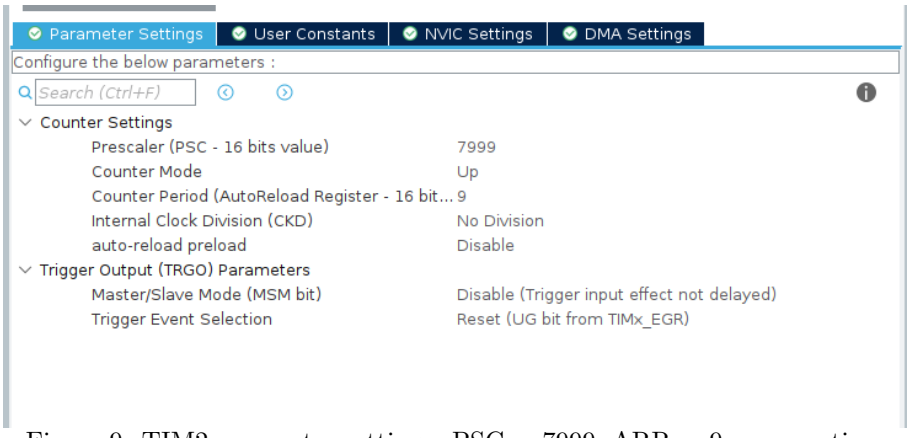


Figure 9: TIM2 parameter settings: PSC = 7999, ARR = 9, up-counting.

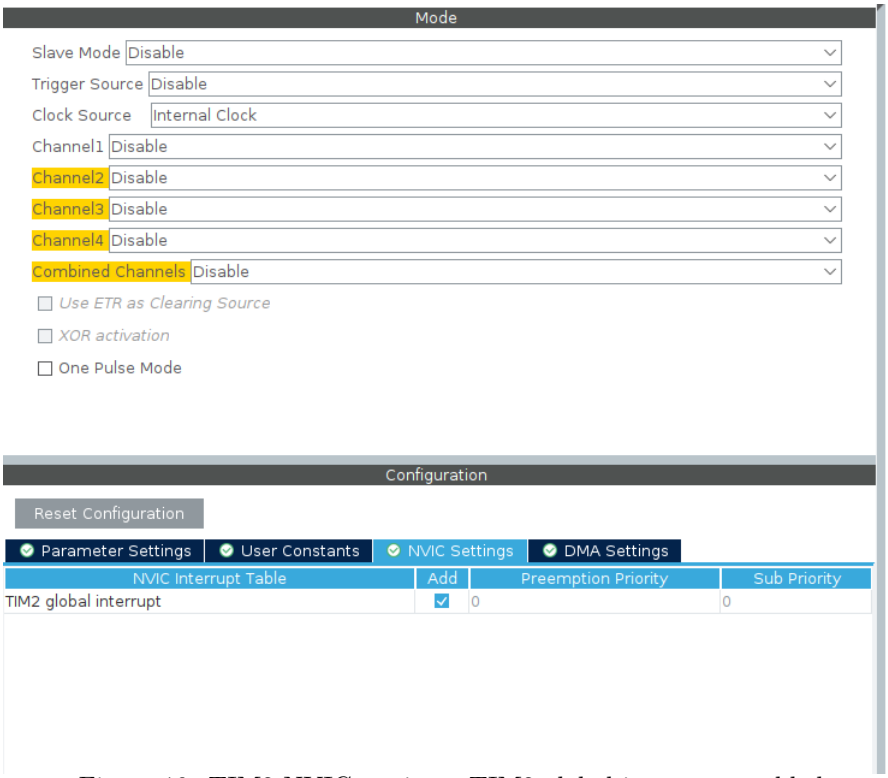


Figure 10: TIM2 NVIC settings: TIM2 global interrupt enabled.

Since the APB1 prescaler is 1 (Figure 8), TIM2 is clocked at $f_{\text{TIM2}} = 8$ MHz. With the internal clock source, $PSC = 7999$, and $ARR = 9$ in up-counting mode (Figure 9):

$$f_{\text{CNT}} = \frac{f_{\text{TIM2}}}{PSC + 1} = \frac{8\,000\,000}{8000} = 1000 \text{ Hz}, \qquad T_{\text{interrupt}} = \frac{ARR + 1}{f_{\text{CNT}}} = \frac{10}{1000} \text{ s} = 10 \text{ ms}.$$

TIM2 therefore generates an update interrupt every 10 ms, that is, at 100 Hz. With the TIM2 global interrupt enabled (Figure 10), each update calls `HAL_TIM_PeriodElapsedCallback()`. This 10-ms interrupt is the time base for button reading and for all software timers in the later exercises, where TIM2 is started with `HAL_TIM_Base_Start_IT(&htim2)`.

Finally, **CMake** is selected as the toolchain in the Project Manager, and the generated project builds in Visual Studio Code without errors.

1.4 Exercise 4: Modify Timer Parameters

1.4.1 Requirement

Your task in this exercise is to modify the timer settings so that when we want to change the time duration of the timer interrupt, we change it the least and it will not affect the overall system. For example, the current system we have implemented is that it can blink an LED in 2 Hz, with the timer interrupt duration is 10ms. However, when we want to change the timer interrupt duration to 1ms or 100ms, it will not affect the 2Hz blinking LED.

1.4.2 Implementation idea

In Lab 02, durations were written as numbers of interrupts, for example `scan_ticks >= 25` for 250 ms. Such constants are valid only for a 10-ms interrupt and must all be changed when the interrupt period changes.

In this exercise, a software-timer module expresses every duration in milliseconds. The interrupt period is defined in exactly one place, `TIMER_CYCLE`, and each TIM2 interrupt subtracts

TIMER_CYCLE from every running timer. The rest of the program never depends on the interrupt period.

1.4.3 Software-timer module

Program 1.2: software_timer.h

```c
#ifndef INC_SOFTWARE_TIMER_H_
#define INC_SOFTWARE_TIMER_H_

#include <stdint.h>

/* TIM2 interrupt period in ms. Must match PSC/ARR in CubeMX. */
#ifndef TIMER_CYCLE
#define TIMER_CYCLE 10
#endif

/* One index per software timer */
enum {
    TIMER_BLINK = 0,   /* Exercise 4: 2 Hz test blink */
    NUM_OF_TIMERS
};

void    setTimer(uint8_t index, uint32_t duration_ms);
uint8_t isTimerExpired(uint8_t index);
void    timerRun(void);

#endif /* INC_SOFTWARE_TIMER_H_ */
```

Program 1.3: software_timer.c

```c
#include "software_timer.h"

/* Remaining time of each timer, in milliseconds */
static volatile int32_t timer_counter[NUM_OF_TIMERS];
/* 1 = the timer has expired */
static volatile uint8_t timer_flag[NUM_OF_TIMERS];

void setTimer(uint8_t index, uint32_t duration_ms)
{
    if (index >= NUM_OF_TIMERS) return;

    int32_t start = (int32_t)duration_ms;

    /* Re-armed after expiring: keep the small overshoot so the
       average period stays exact even if TIMER_CYCLE does not
       divide duration_ms. */
    if (timer_flag[index]) {
        start += timer_counter[index];   /* counter is <= 0 here */
    }

    timer_flag[index]   = 0;
    timer_counter[index] = start;

```

```
    if (start <= 0) {
        timer_flag[index] = 1;           /* already due */
    }
}

uint8_t isTimerExpired(uint8_t index)
{
    if (index >= NUM_OF_TIMERS) return 0;
    return timer_flag[index];
}

void timerRun(void)
{
    for (uint8_t i = 0; i < NUM_OF_TIMERS; i++) {
        if (timer_counter[i] > 0) {
            timer_counter[i] -= TIMER_CYCLE;
            if (timer_counter[i] <= 0) {
                timer_flag[i] = 1;
            }
        }
    }
}
```

- `setTimer(index, duration_ms)` starts timer `index` with a duration in milliseconds and clears its flag.
- `timerRun()` is called in every TIM2 interrupt and subtracts `TIMER_CYCLE` from each running timer; when a timer reaches zero, its flag is set.
- `isTimerExpired(index)` returns 1 when the timer has expired. The flag is cleared by the next `setTimer()`.

The counters are `volatile` because they are modified inside the interrupt. They are signed: if `TIMER_CYCLE` does not divide the duration, the counter may end slightly below zero, and `setTimer()` carries this overshoot into the next period so that the average period remains exact.

1.4.4 Usage in main.c

The 2-Hz test toggles all red LEDs every 250 ms, giving one 500-ms on–off cycle.

Program 1.4: 2-Hz blinking test in main.c

```
/* USER CODE BEGIN 2 */
setTimer(TIMER_BLINK, 250);        /* first toggle after 250 ms */
HAL_TIM_Base_Start_IT(&htim2);     /* start the 10 ms interrupt */
/* USER CODE END 2 */

while (1) {
    if (isTimerExpired(TIMER_BLINK)) {
        HAL_GPIO_TogglePin(GPIOA, red_h_Pin | red_v_Pin);
        setTimer(TIMER_BLINK, 250);
    }
}

```

```
13 /* USER CODE BEGIN 4 */
14 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
15 {
16     if (htim->Instance == TIM2) {
17         timerRun();
18     }
19 }
20 /* USER CODE END 4 */
```

The new source file is added to `target_sources` in the top-level `CMakeLists.txt`.

1.4.5 Changing the interrupt period

Only two settings change: PSC/ARR in STM32CubeMX and `TIMER_CYCLE`. The application code is unchanged.

Table 13: Settings for different interrupt periods

Interrupt period	PSC	ARR	TIMER_CYCLE	Blink frequency
1 ms	799	9	1	2 Hz
10 ms	7999	9	10	2 Hz
100 ms	7999	99	100	2 Hz (average)

For 1 ms, PSC is reduced to 799 instead of setting ARR to 0, because a zero auto-reload value stops the STM32 counter.

1.4.6 Result

In Proteus, the four red LEDs blink together at 2 Hz, and all other LEDs and digits remain off. A 60-second software simulation of the module gives 240 toggles, that is, exactly 2 Hz, for all three interrupt periods. With 100 ms, a 250-ms toggle cannot fall exactly on an interrupt, so individual toggles alternate between 300 ms and 200 ms; the carried overshoot keeps the average at 2 Hz. A software timer cannot be more precise than one interrupt period.

1.5 Exercise 5: Adding Code for Button Debouncing

1.5.1 Requirement

Following the example of button reading and debouncing in the previous section, your tasks in this exercise are:

- To add new files for input reading and output display,
- To add code for button debouncing,
- To add code for increasing mode when the first button is pressed.

1.5.2 Implementation

The code is divided into modules, following Figure 1.5 of the lab manual.

Table 14: Files changed or added in Exercise 5

File	Function	Called from
`software_timer.h` (updated)	Adds one auto-repeat timer per button	–
`input_reading.h/.c` (new)	Reads and debounces the three buttons; detects a 1-s hold	TIM2 interrupt
`input_processing.h/.c` (new)	Button FSM and mode FSM of Exercise 1	Main loop
`led_display.h/.c` (new)	Turns on one colour per road	Main loop
`main.c`, `CMakeLists.txt`	Calls the modules; adds the new sources to the build	–

1.5.2.1 Software timer. `software_timer.c` is unchanged from Exercise 4. The header adds one timer per button for the 500-ms auto-repeat of the button FSM.

Program 1.5: software_timer.h (updated)

```
#ifndef INC_SOFTWARE_TIMER_H_
#define INC_SOFTWARE_TIMER_H_

#include <stdint.h>

/* TIM2 interrupt period in ms. Must match PSC/ARR in CubeMX. */
#ifndef TIMER_CYCLE
#define TIMER_CYCLE 10
#endif

/* One index per software timer */
enum {
    TIMER_BLINK = 0,      /* 2 Hz blinking (used again in Exercise 6) */
    TIMER_BTN0_REPEAT,    /* auto-repeat of button 0 when held      */
    TIMER_BTN1_REPEAT,    /* auto-repeat of button 1 when held      */
    TIMER_BTN2_REPEAT,    /* auto-repeat of button 2 when held      */
    NUM_OF_TIMERS
};

void    setTimer(uint8_t index, uint32_t duration_ms);
uint8_t isTimerExpired(uint8_t index);
void    timerRun(void);

#endif /* INC_SOFTWARE_TIMER_H_ */
```

1.5.2.2 Input reading and debouncing. `button_reading()` runs every 10 ms in the TIM2 interrupt. A new level is accepted only when two consecutive readings are equal, which filters out contact bouncing. The 1-s threshold is `1000 / TIMER_CYCLE` interrupts, so it remains correct if the interrupt period changes (Exercise 4). Unlike the example in the lab manual, the buffers start as *released*: a zero initial value would mean *pressed* and cause a false mode change at reset.

Program 1.6: input_reading.h

```c
#ifndef INC_INPUT_READING_H_
#define INC_INPUT_READING_H_

#include <stdint.h>

#define NO_OF_BUTTONS 3

/* Button indexes */
#define BUTTON_MODE  0   /* sw0 - PA11 */
#define BUTTON_INC   1   /* sw1 - PA12 */
#define BUTTON_SET   2   /* sw2 - PA13 */

void    button_reading(void);
uint8_t is_button_pressed(uint8_t index);
uint8_t is_button_pressed_1s(uint8_t index);

#endif /* INC_INPUT_READING_H_ */
```

Program 1.7: input_reading.c

```c
#include "main.h"
#include "input_reading.h"
#include "software_timer.h"

/* Held for 1 s, expressed in interrupts (independent of TIMER_CYCLE) */
#define DURATION_FOR_1S     (1000 / TIMER_CYCLE)

#define BUTTON_IS_PRESSED   GPIO_PIN_RESET
#define BUTTON_IS_RELEASED  GPIO_PIN_SET

/* Port and pin of each button, in index order */
static GPIO_TypeDef *const button_port[NO_OF_BUTTONS] =
    { sw0_GPIO_Port, sw1_GPIO_Port, sw2_GPIO_Port };
static const uint16_t button_pin[NO_OF_BUTTONS] =
    { sw0_Pin, sw1_Pin, sw2_Pin };

/* Final debounced state. Start as RELEASED to avoid a false press. */
static GPIO_PinState buttonBuffer[NO_OF_BUTTONS] =
    { BUTTON_IS_RELEASED, BUTTON_IS_RELEASED, BUTTON_IS_RELEASED };

/* The two most recent raw readings */
static GPIO_PinState debounceButtonBuffer1[NO_OF_BUTTONS] =
    { BUTTON_IS_RELEASED, BUTTON_IS_RELEASED, BUTTON_IS_RELEASED };
static GPIO_PinState debounceButtonBuffer2[NO_OF_BUTTONS] =
    { BUTTON_IS_RELEASED, BUTTON_IS_RELEASED, BUTTON_IS_RELEASED };

/* Long-press detection */
static uint8_t  flagForButtonPress1s[NO_OF_BUTTONS];
static uint16_t counterForButtonPress1s[NO_OF_BUTTONS];

/* Called every TIMER_CYCLE ms from the TIM2 interrupt */
void button_reading(void)
```

```
{
    for (uint8_t i = 0; i < NO_OF_BUTTONS; i++) {
        debounceButtonBuffer2[i] = debounceButtonBuffer1[i];
        debounceButtonBuffer1[i] = HAL_GPIO_ReadPin(button_port[i],
                                                    button_pin[i]);

        /* Accept the new level only if two readings agree */
        if (debounceButtonBuffer1[i] == debounceButtonBuffer2[i]) {
            buttonBuffer[i] = debounceButtonBuffer1[i];
        }

        if (buttonBuffer[i] == BUTTON_IS_PRESSED) {
            if (counterForButtonPress1s[i] < DURATION_FOR_1S) {
                counterForButtonPress1s[i]++;
            } else {
                flagForButtonPress1s[i] = 1;
            }
        } else {
            counterForButtonPress1s[i] = 0;
            flagForButtonPress1s[i] = 0;
        }
    }
}

uint8_t is_button_pressed(uint8_t index)
{
    if (index >= NO_OF_BUTTONS) return 0;
    return (buttonBuffer[index] == BUTTON_IS_PRESSED);
}

uint8_t is_button_pressed_1s(uint8_t index)
{
    if (index >= NO_OF_BUTTONS) return 0;
    return (flagForButtonPress1s[index] == 1);
}
```

1.5.2.3 Input processing and increasing the mode.

`fsm_button()` implements the button FSM of Figure 1. It sets `pressEvent[i]` only when a button enters `BUTTON_PRESSED`, so one physical press produces exactly one event; `repeatEvent[i]` is set every 500 ms while the button is held. `take()` reads an event and clears it, so each event is handled once. The mode FSM uses the press event of the first button to step through Modes $1 \to 2 \to 3 \to 4 \to 1$; its auto-repeat event is discarded, so holding the button changes the mode only once.

Program 1.8: input_processing.h

```
#ifndef INC_INPUT_PROCESSING_H_
#define INC_INPUT_PROCESSING_H_

#include <stdint.h>

/* Modes of the application */
enum {
    MODE_NORMAL = 1,   /* Mode 1 */
```

```
    MODE_RED,           /* Mode 2 */
    MODE_AMBER,         /* Mode 3 */
    MODE_GREEN          /* Mode 4 */
};

void    fsm_for_input_processing(void);
uint8_t get_mode(void);

#endif /* INC_INPUT_PROCESSING_H_ */
```

Program 1.9: input_processing.c

```
#include "main.h"
#include "input_reading.h"
#include "input_processing.h"
#include "software_timer.h"

#define AUTO_REPEAT_MS 500

/* ---------- FSM A: one copy per button (Exercise 1) ---------- */
enum ButtonState {
    BUTTON_RELEASED,
    BUTTON_PRESSED,
    BUTTON_PRESSED_MORE_THAN_1_SECOND
};

static enum ButtonState buttonState[NO_OF_BUTTONS];   /* all RELEASED */

/* One-shot events produced by the FSM */
static uint8_t pressEvent[NO_OF_BUTTONS];    /* "output +1" on a new press */
static uint8_t repeatEvent[NO_OF_BUTTONS];   /* "+1 every 500 ms" when held */

static void fsm_button(uint8_t i)
{
    switch (buttonState[i]) {
    case BUTTON_RELEASED:
        if (is_button_pressed(i)) {
            buttonState[i] = BUTTON_PRESSED;
            pressEvent[i] = 1;                          /* output +1 */
        }
        break;

    case BUTTON_PRESSED:
        if (!is_button_pressed(i)) {
            buttonState[i] = BUTTON_RELEASED;
        } else if (is_button_pressed_1s(i)) {
            buttonState[i] = BUTTON_PRESSED_MORE_THAN_1_SECOND;
            setTimer(TIMER_BTN0_REPEAT + i, AUTO_REPEAT_MS);
        }
        break;

    case BUTTON_PRESSED_MORE_THAN_1_SECOND:
        if (!is_button_pressed(i)) {
            buttonState[i] = BUTTON_RELEASED;
```

```
        } else if (isTimerExpired(TIMER_BTN0_REPEAT + i)) {
            repeatEvent[i] = 1;                          /* +1 every 500 ms */
            setTimer(TIMER_BTN0_REPEAT + i, AUTO_REPEAT_MS);
        }
        break;
    }
}

/* Read and clear an event (returns 1 once per event) */
static uint8_t take(uint8_t *event)
{
    uint8_t value = *event;
    *event = 0;
    return value;
}

/* ---------- FSM B: mode selection ---------- */
static uint8_t mode = MODE_NORMAL;

void fsm_for_input_processing(void)
{
    for (uint8_t i = 0; i < NO_OF_BUTTONS; i++) {
        fsm_button(i);
    }

    /* B1: one press = next mode, 1 -> 2 -> 3 -> 4 -> 1 */
    if (take(&pressEvent[BUTTON_MODE])) {
        mode++;
        if (mode > MODE_GREEN) {
            mode = MODE_NORMAL;
        }
    }
    repeatEvent[BUTTON_MODE] = 0;   /* holding B1 never changes the mode */
}

uint8_t get_mode(void)
{
    return mode;
}
```

1.5.2.4 Output display. `setLightH()` and `setLightV()` first turn all three colours of a road off and then turn on only the selected one, so two colours of the same road are never on together. The LEDs are active low.

Program 1.10: led_display.h

```
#ifndef INC_LED_DISPLAY_H_
#define INC_LED_DISPLAY_H_

#include <stdint.h>

/* Colour shown by one road */
enum {
    LIGHT_OFF = 0,
```

```
    LIGHT_RED,
    LIGHT_AMBER,
    LIGHT_GREEN
};

void setLightH(uint8_t light);   /* horizontal road: D1-D6  */
void setLightV(uint8_t light);   /* vertical road:   D7-D12 */

#endif /* INC_LED_DISPLAY_H_ */
```

Program 1.11: led_display.c

```
#include "main.h"
#include "led_display.h"

/* LEDs are active low: RESET = ON, SET = OFF */

void setLightH(uint8_t light)
{
    /* Turn the three colours off first */
    HAL_GPIO_WritePin(GPIOA, red_h_Pin | yel_h_Pin | grn_h_Pin,
                      GPIO_PIN_SET);

    if (light == LIGHT_RED)   HAL_GPIO_WritePin(red_h_GPIO_Port, red_h_Pin,
    GPIO_PIN_RESET);
    if (light == LIGHT_AMBER) HAL_GPIO_WritePin(yel_h_GPIO_Port, yel_h_Pin,
    GPIO_PIN_RESET);
    if (light == LIGHT_GREEN) HAL_GPIO_WritePin(grn_h_GPIO_Port, grn_h_Pin,
    GPIO_PIN_RESET);
}

void setLightV(uint8_t light)
{
    HAL_GPIO_WritePin(GPIOA, red_v_Pin | yel_v_Pin | grn_v_Pin,
                      GPIO_PIN_SET);

    if (light == LIGHT_RED)   HAL_GPIO_WritePin(red_v_GPIO_Port, red_v_Pin,
    GPIO_PIN_RESET);
    if (light == LIGHT_AMBER) HAL_GPIO_WritePin(yel_v_GPIO_Port, yel_v_Pin,
    GPIO_PIN_RESET);
    if (light == LIGHT_GREEN) HAL_GPIO_WritePin(grn_v_GPIO_Port, grn_v_Pin,
    GPIO_PIN_RESET);
}
```

1.5.2.5 Main program and build. The interrupt only updates the timers and reads the buttons; all decisions are made in the main loop. For testing, the current mode is shown on the traffic LEDs: off in Mode 1, and red, amber, or green in Modes 2, 3, and 4. The LEDs are written only when the mode changes.

Program 1.12: Changes in main.c (USER CODE sections)

```
/* USER CODE BEGIN Includes */
#include "software_timer.h"
#include "input_reading.h"
#include "input_processing.h"
#include "led_display.h"
/* USER CODE END Includes */

  /* USER CODE BEGIN 2 */
  HAL_TIM_Base_Start_IT(&htim2);
  uint8_t last_mode = 0;          /* 0 = force the first LED update */
  /* USER CODE END 2 */

  /* USER CODE BEGIN WHILE */
  while (1)
  {
    fsm_for_input_processing();

    /* Exercise 5 test: show the current mode on the traffic LEDs
       (Mode 1 = off, 2 = red, 3 = amber, 4 = green) */
    uint8_t mode = get_mode();
    if (mode != last_mode) {
      last_mode = mode;
      uint8_t light = LIGHT_OFF;
      if (mode == MODE_RED)   light = LIGHT_RED;
      if (mode == MODE_AMBER) light = LIGHT_AMBER;
      if (mode == MODE_GREEN) light = LIGHT_GREEN;
      setLightH(light);
      setLightV(light);
    }
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
  }
  /* USER CODE END 3 */

/* USER CODE BEGIN 4 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
  if (htim->Instance == TIM2) {
    timerRun();
    button_reading();
  }
}
/* USER CODE END 4 */
```

Program 1.13: Sources added to the top-level CMakeLists.txt

```
target_sources(${CMAKE_PROJECT_NAME} PRIVATE
    # Add user sources here
    Core/Src/software_timer.c
    Core/Src/input_reading.c
    Core/Src/input_processing.c
```

```
    Core/Src/led_display.c
)
```

1.5.3 Result

Table 15: Test results of Exercise 5

Test	Result
Reset, no button pressed	Mode 1, all LEDs off
Press B1 four times	Modes 2, 3, 4, 1: red, amber, green, off
Hold B1 for 3 s	The mode advances only once
Press B2 or B3	No change
Press B1 quickly and repeatedly	Exactly one step per press

All tests pass in Proteus. Every press of the first button increases the mode by exactly one, so the bouncing of the contacts is fully filtered.

1.6 Exercise 6: Adding Code for Displaying Modes

1.6.1 Requirement

Your tasks in this exercise are:

- To add code for display mode on seven-segment LEDs, and
- To add code for blinking LEDs depending on the mode that is selected.

1.6.2 Implementation

Exercise 5 already selects the mode with B1; this exercise makes the selected mode visible. Following Figure 1.5 of the lab manual, the output side is divided into three layers, so the display logic never accesses the GPIO pins directly.

Table 16: Output layers added in Exercise 6

Layer	Code	Task
Display logic	`display_mode()` in `input_processing.c`	Decides what is shown in the current mode
Output buffer	`led_buffer[4]`, `set7SegH()`, `set7SegV()`	Holds the four digits to be displayed
Output driver	`scan7Seg()`, `setLightH()`, `setLightV()`	Writes the digits and colours to the GPIO pins

Table 17 shows the output of each mode. The two digits of road H show the value of the edited duration, and the two digits of road V show the mode number. The traffic light of Mode 1 is added in Exercise 10; until then, Mode 1 only shows its number.

Table 17: Display of each mode

Mode	Blinking LEDs (2 Hz)	Road H (en0, en1)	Road V (en2, en3)
1	None (all off)	Blank	01
2	Red (D1, D4, D7, D10)	05 (T_red)	02
3	Amber (D2, D5, D8, D11)	02 (T_amber)	03
4	Green (D3, D6, D9, D12)	03 (T_green)	04

1.6.2.1 Seven-segment scanning. The two digits of one road share a segment bus, so they are switched on alternately. Because roads H and V have separate buses (Exercise 2), one scan needs only two steps: the tens digits of both roads (en0, en2), then the units digits (en1, en3). `scan7Seg()` performs one step each time `TIMER_SCAN` expires. It first turns all four digits off, which prevents ghosting, then writes the segments and enables the next pair. With 10 ms per step, a full scan takes 20 ms (50 Hz), so the display looks steady. `SEG_CODE` stores the segment pattern of each digit (bit 0 = segment a, . . . , bit 6 = segment g); since the display is common-anode, a lit segment is written as `GPIO_PIN_RESET`.

1.6.2.2 Blinking at 2 Hz. A 2-Hz blink is one 500-ms on–off cycle, so the LEDs toggle every 250 ms using `TIMER_BLINK`. `display_mode()` handles two cases:

- **A new mode is entered:** the digits are updated once, the selected colour is turned on, and the blink timer restarts;
- **The mode is unchanged:** each time the blink timer expires, `blink_on` is inverted and the LEDs are updated.

The GPIO pins are therefore written only when something changes. All durations are in milliseconds, so the blinking stays at 2 Hz when the interrupt period changes (Exercise 4).

1.6.2.3 Durations. The default durations are T_red = 5 s, T_amber = 2 s, and T_green = 3 s, as in the example of Exercise 1; they satisfy $T_{\text{red}} = T_{\text{green}} + T_{\text{amber}}$. When a mode is entered, `load_temp()` copies the duration of the selected colour into `temp_value`, which is displayed now and edited in Exercises 7–9.

Table 18: Files changed in Exercise 6

File	Change
`software_timer.h`	Adds `TIMER_SCAN`
`led_display.h/.c`	Adds the seven-segment buffer and scanning
`input_processing.h/.c`	Adds the durations, `load_temp()`, and `display_mode()`
`main.c`	Calls `display_mode()` and `scan7Seg()`; removes the Exercise 5 test

`software_timer.c` and `input_reading.h/.c` are unchanged. No source file is added, so `CMakeLists.txt` is also unchanged.

Program 1.14: software_timer.h (updated)

```
1  #ifndef INC_SOFTWARE_TIMER_H_
2  #define INC_SOFTWARE_TIMER_H_
3
4  #include <stdint.h>
5
```

```
/* TIM2 interrupt period in ms. Must match PSC/ARR in CubeMX. */
#ifndef TIMER_CYCLE
#define TIMER_CYCLE 10
#endif

/* One index per software timer */
enum {
    TIMER_BLINK = 0,      /* 2 Hz blinking in Modes 2-4            */
    TIMER_BTN0_REPEAT,    /* auto-repeat of button 0 when held     */
    TIMER_BTN1_REPEAT,    /* auto-repeat of button 1 when held     */
    TIMER_BTN2_REPEAT,    /* auto-repeat of button 2 when held     */
    TIMER_SCAN,           /* Exercise 6: 7-segment scanning        */
    NUM_OF_TIMERS
};

void    setTimer(uint8_t index, uint32_t duration_ms);
uint8_t isTimerExpired(uint8_t index);
void    timerRun(void);

#endif /* INC_SOFTWARE_TIMER_H_ */
```

Program 1.15: led_display.h (updated)

```
#ifndef INC_LED_DISPLAY_H_
#define INC_LED_DISPLAY_H_

#include <stdint.h>

/* Colour shown by one road */
enum {
    LIGHT_OFF = 0,
    LIGHT_RED,
    LIGHT_AMBER,
    LIGHT_GREEN
};

/* Value > 99 turns the two digits of a road off */
#define SEG_BLANK 0xFF

void setLightH(uint8_t light);   /* horizontal road: D1-D6  */
void setLightV(uint8_t light);   /* vertical road:   D7-D12 */

void set7SegH(uint8_t value);    /* 2 digits of road H (en0, en1) */
void set7SegV(uint8_t value);    /* 2 digits of road V (en2, en3) */
void scan7Seg(void);             /* show the next digit pair      */

#endif /* INC_LED_DISPLAY_H_ */
```

Program 1.16: led_display.c (updated)

```
#include "main.h"
```

```c
#include "led_display.h"

/* Everything is active low: RESET = ON, SET = OFF */

/* ---------------- Traffic LEDs (unchanged from Exercise 5) ---------------- */
void setLightH(uint8_t light)
{
    HAL_GPIO_WritePin(GPIOA, red_h_Pin | yel_h_Pin | grn_h_Pin, GPIO_PIN_SET);

    if (light == LIGHT_RED)   HAL_GPIO_WritePin(red_h_GPIO_Port, red_h_Pin,
    GPIO_PIN_RESET);
    if (light == LIGHT_AMBER) HAL_GPIO_WritePin(yel_h_GPIO_Port, yel_h_Pin,
    GPIO_PIN_RESET);
    if (light == LIGHT_GREEN) HAL_GPIO_WritePin(grn_h_GPIO_Port, grn_h_Pin,
    GPIO_PIN_RESET);
}

void setLightV(uint8_t light)
{
    HAL_GPIO_WritePin(GPIOA, red_v_Pin | yel_v_Pin | grn_v_Pin, GPIO_PIN_SET);

    if (light == LIGHT_RED)   HAL_GPIO_WritePin(red_v_GPIO_Port, red_v_Pin,
    GPIO_PIN_RESET);
    if (light == LIGHT_AMBER) HAL_GPIO_WritePin(yel_v_GPIO_Port, yel_v_Pin,
    GPIO_PIN_RESET);
    if (light == LIGHT_GREEN) HAL_GPIO_WritePin(grn_v_GPIO_Port, grn_v_Pin,
    GPIO_PIN_RESET);
}

/* ---------------- Seven-segment LEDs (Exercise 6) ---------------- */
#define DIGIT_BLANK 10

/* Bit 0 = segment a ... bit 6 = segment g; 1 = segment lit */
static const uint8_t SEG_CODE[11] = {
    0x3F, 0x06, 0x5B, 0x4F, 0x66,   /* 0 1 2 3 4 */
    0x6D, 0x7D, 0x07, 0x7F, 0x6F,   /* 5 6 7 8 9 */
    0x00                            /* blank     */
};

/* Segment pins a..g of each road (all on GPIOB) */
static const uint16_t seg_pin_h[7] = {
    a_h_Pin, b_h_Pin, c_h_Pin, d_h_Pin, e_h_Pin, f_h_Pin, g_h_Pin };
static const uint16_t seg_pin_v[7] = {
    a_v_Pin, b_v_Pin, c_v_Pin, d_v_Pin, e_v_Pin, f_v_Pin, g_v_Pin };

/* Output buffer: {H tens, H units, V tens, V units} */
static uint8_t led_buffer[4] = {
    DIGIT_BLANK, DIGIT_BLANK, DIGIT_BLANK, DIGIT_BLANK };

static uint8_t scan_step = 0;   /* 0 = tens digits, 1 = units digits */

/* Light the segments of one digit on one bus */
static void writeSegments(const uint16_t pin[7], uint8_t digit)
{
    uint8_t code = SEG_CODE[digit];
    for (uint8_t i = 0; i < 7; i++) {
```

```
52          if (code & (1 << i)) {
53              HAL_GPIO_WritePin(GPIOB, pin[i], GPIO_PIN_RESET);   /* on  */
54          } else {
55              HAL_GPIO_WritePin(GPIOB, pin[i], GPIO_PIN_SET);     /* off */
56          }
57      }
58  }
59
60  void set7SegH(uint8_t value)
61  {
62      if (value > 99) {
63          led_buffer[0] = DIGIT_BLANK;
64          led_buffer[1] = DIGIT_BLANK;
65      } else {
66          led_buffer[0] = value / 10;    /* tens  */
67          led_buffer[1] = value % 10;    /* units */
68      }
69  }
70
71  void set7SegV(uint8_t value)
72  {
73      if (value > 99) {
74          led_buffer[2] = DIGIT_BLANK;
75          led_buffer[3] = DIGIT_BLANK;
76      } else {
77          led_buffer[2] = value / 10;
78          led_buffer[3] = value % 10;
79      }
80  }
81
82  /* Called every SCAN_MS: digit 1 of both roads, then digit 2 of both roads */
83  void scan7Seg(void)
84  {
85      /* 1. Turn all 4 digits off first (no ghosting) */
86      HAL_GPIO_WritePin(GPIOA, en0_Pin | en1_Pin | en2_Pin | en3_Pin, GPIO_PIN_SET);
87
88      if (scan_step == 0) {
89          /* 2. Tens digits of both roads */
90          writeSegments(seg_pin_h, led_buffer[0]);
91          writeSegments(seg_pin_v, led_buffer[2]);
92          HAL_GPIO_WritePin(GPIOA, en0_Pin | en2_Pin, GPIO_PIN_RESET);
93          scan_step = 1;
94      } else {
95          /* 2. Units digits of both roads */
96          writeSegments(seg_pin_h, led_buffer[1]);
97          writeSegments(seg_pin_v, led_buffer[3]);
98          HAL_GPIO_WritePin(GPIOA, en1_Pin | en3_Pin, GPIO_PIN_RESET);
99          scan_step = 0;
100     }
101 }
```

Program 1.17: input_processing.h (updated)

```
#ifndef INC_INPUT_PROCESSING_H_
#define INC_INPUT_PROCESSING_H_

#include <stdint.h>

/* Modes of the application */
enum {
    MODE_NORMAL = 1,    /* Mode 1 */
    MODE_RED,           /* Mode 2 */
    MODE_AMBER,         /* Mode 3 */
    MODE_GREEN          /* Mode 4 */
};

void    fsm_for_input_processing(void);
void    display_mode(void);          /* Exercise 6 */
uint8_t get_mode(void);

#endif /* INC_INPUT_PROCESSING_H_ */
```

Program 1.18: input_processing.c (updated)

```
#include "main.h"
#include "input_reading.h"
#include "input_processing.h"
#include "software_timer.h"
#include "led_display.h"

#define AUTO_REPEAT_MS 500
#define BLINK_MS       250     /* toggle every 250 ms = 2 Hz */

/* ---------- FSM A: one copy per button (unchanged from Exercise 5) ---------- */
enum ButtonState {
    BUTTON_RELEASED,
    BUTTON_PRESSED,
    BUTTON_PRESSED_MORE_THAN_1_SECOND
};

static enum ButtonState buttonState[NO_OF_BUTTONS];
static uint8_t pressEvent[NO_OF_BUTTONS];
static uint8_t repeatEvent[NO_OF_BUTTONS];

static void fsm_button(uint8_t i)
{
    switch (buttonState[i]) {
    case BUTTON_RELEASED:
        if (is_button_pressed(i)) {
            buttonState[i] = BUTTON_PRESSED;
            pressEvent[i] = 1;
        }
        break;

    case BUTTON_PRESSED:
```

```
32          if (!is_button_pressed(i)) {
33              buttonState[i] = BUTTON_RELEASED;
34          } else if (is_button_pressed_1s(i)) {
35              buttonState[i] = BUTTON_PRESSED_MORE_THAN_1_SECOND;
36              setTimer(TIMER_BTN0_REPEAT + i, AUTO_REPEAT_MS);
37          }
38          break;
39
40      case BUTTON_PRESSED_MORE_THAN_1_SECOND:
41          if (!is_button_pressed(i)) {
42              buttonState[i] = BUTTON_RELEASED;
43          } else if (isTimerExpired(TIMER_BTN0_REPEAT + i)) {
44              repeatEvent[i] = 1;
45              setTimer(TIMER_BTN0_REPEAT + i, AUTO_REPEAT_MS);
46          }
47          break;
48      }
49  }
50
51  static uint8_t take(uint8_t *event)
52  {
53      uint8_t value = *event;
54      *event = 0;
55      return value;
56  }
57
58  /* ---------- Durations (seconds) and the edited value ---------- */
59  static uint8_t time_red   = 5;
60  static uint8_t time_amber = 2;
61  static uint8_t time_green = 3;
62  static uint8_t temp_value = 0;   /* value shown/edited in Modes 2-4 */
63
64  /* Copy the duration of the selected colour into temp_value */
65  static void load_temp(void)
66  {
67      if (get_mode() == MODE_RED)   temp_value = time_red;
68      if (get_mode() == MODE_AMBER) temp_value = time_amber;
69      if (get_mode() == MODE_GREEN) temp_value = time_green;
70  }
71
72  /* ---------- FSM B: mode selection ---------- */
73  static uint8_t mode = MODE_NORMAL;
74
75  void fsm_for_input_processing(void)
76  {
77      for (uint8_t i = 0; i < NO_OF_BUTTONS; i++) {
78          fsm_button(i);
79      }
80
81      /* B1: one press = next mode, 1 -> 2 -> 3 -> 4 -> 1 */
82      if (take(&pressEvent[BUTTON_MODE])) {
83          mode++;
84          if (mode > MODE_GREEN) {
85              mode = MODE_NORMAL;
86          }
87          load_temp();              /* Exercise 6: value of the new mode */
```

```
    }
    repeatEvent[BUTTON_MODE] = 0;
}

uint8_t get_mode(void)
{
    return mode;
}

/* ---------- Exercise 6: display the mode ---------- */
static uint8_t shown_mode = 0;   /* 0 = nothing shown yet */
static uint8_t blink_on   = 0;   /* 1 = blinking LEDs are on now */

/* Colour that blinks in the current mode */
static uint8_t mode_colour(void)
{
    if (mode == MODE_RED)   return LIGHT_RED;
    if (mode == MODE_AMBER) return LIGHT_AMBER;
    if (mode == MODE_GREEN) return LIGHT_GREEN;
    return LIGHT_OFF;             /* Mode 1: traffic FSM added in Exercise 10 */
}

static void update_lights(void)
{
    uint8_t light = blink_on ? mode_colour() : LIGHT_OFF;
    setLightH(light);
    setLightV(light);
}

void display_mode(void)
{
    if (mode != shown_mode) {
        /* A new mode was entered: update everything once */
        shown_mode = mode;

        if (mode == MODE_NORMAL) {
            set7SegH(SEG_BLANK);     /* no value in Mode 1 (yet) */
        } else {
            set7SegH(temp_value);    /* road H: the value  */
        }
        set7SegV(mode);              /* road V: the mode, e.g. 02 */

        blink_on = 1;                /* start with the LEDs on */
        setTimer(TIMER_BLINK, BLINK_MS);
        update_lights();
    }
    else if (mode != MODE_NORMAL && isTimerExpired(TIMER_BLINK)) {
        /* Same mode: toggle the colour every 250 ms (2 Hz) */
        blink_on = !blink_on;
        setTimer(TIMER_BLINK, BLINK_MS);
        update_lights();
    }
}
```

1.6.2.4 Main program.

`TIMER_SCAN` must be started once before the loop; a timer that has never been set never expires. The test display of Exercise 5 is replaced by `display_mode()`, and

the interrupt callback is unchanged.

Program 1.19: Changes in main.c (USER CODE sections)

```
/* USER CODE BEGIN Includes */
#include "software_timer.h"
#include "input_reading.h"
#include "input_processing.h"
#include "led_display.h"
/* USER CODE END Includes */

/* USER CODE BEGIN PD */
#define SCAN_MS 10          /* one digit pair every 10 ms -> full scan 20 ms (50
    Hz) */
/* USER CODE END PD */

  /* USER CODE BEGIN 2 */
  HAL_TIM_Base_Start_IT(&htim2);
  setTimer(TIMER_SCAN, SCAN_MS);   /* must be armed once, or it never expires */
  /* USER CODE END 2 */

  /* USER CODE BEGIN WHILE */
  while (1)
  {
    fsm_for_input_processing();    /* buttons + mode FSM          */
    display_mode();                /* Ex6: 7-seg buffer + blinking */

    if (isTimerExpired(TIMER_SCAN)) {
      setTimer(TIMER_SCAN, SCAN_MS);
      scan7Seg();                  /* show the next digit pair    */
    }
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
  }
  /* USER CODE END 3 */

/* USER CODE BEGIN 4 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
  if (htim->Instance == TIM2) {
    timerRun();
    button_reading();
  }
}
/* USER CODE END 4 */
```

1.6.3 Result

Table 19: Test results of Exercise 6

Test	Result
Reset, no button pressed	Mode 1: LEDs off, H blank, V = 01
Press B1 once	Mode 2: red LEDs blink, H = 05, V = 02
Press B1 again	Mode 3: amber LEDs blink, H = 02, V = 03
Press B1 again	Mode 4: green LEDs blink, H = 03, V = 04
Press B1 again	Back to Mode 1
Hold B1 for 3 s	The mode advances only once
Press B2 or B3	No change (used from Exercise 7)

In every edit mode, only the selected colour blinks, and both roads blink together. A software simulation of the firmware with a mocked HAL gives exactly 12 toggles in 3 s, that is 2 Hz, for interrupt periods of 1, 10, and 100 ms; it also confirms that the two digits of one road are never enabled at the same time. With a 100-ms interrupt, however, a full scan takes 200 ms and the display visibly flickers, so the interrupt period should stay at 10 ms or below for the seven-segment LEDs.

1.7 Exercise 7: Adding Code for Increasing Time Duration Value for the Red LEDs

1.7.1 Requirement

Your tasks in this exercise are:

- to use the second button to increase the time duration value of the red LEDs
- to use the third button to set the value for the red LEDs.

1.7.2 Implementation

Only `input_processing.c` changes in this exercise; all other files, including `main.c` and `CMakeLists.txt`, are the same as in Exercise 6. The new code implements the self-loops of Mode 2 in the mode FSM (Figure 2).

1.7.2.1 Reading B2 and B3. The events of B2 and B3 are taken in every mode but used only in Mode 2. Otherwise, a press made in another mode would remain stored and would be applied as soon as Mode 2 is entered. B2 combines two events: a single press, and the auto-repeat of FSM A, which adds 1 every 500 ms while the button is held for more than one second. Holding B3 saves only once.

1.7.2.2 Increasing the value. `increase_temp()` adds 1 to `temp_value` and wraps from 99 back to 1, which keeps the value inside the range 1–99 required by the specification. Only the draft `temp_value` changes; the real duration `time_red` is not modified until B3 is pressed.

1.7.2.3 Setting the value. The lab manual does not state how the three durations are related. This design keeps the rule of Exercise 1, $T_{\text{red}} = T_{\text{green}} + T_{\text{amber}}$, so that the red countdown always matches the time for which the red light is on. When B3 is pressed, `save_red()` therefore:

- saves `temp_value` as T_red and sets T_green = T_red − T_amber, if T_red > T_amber;
- otherwise rejects the value, because T_green would be 0, and restores the saved red duration on the display.

The amber duration is unchanged. If B1 is pressed without B3, `load_temp()` reloads the saved value in the next mode, so the draft is discarded.

1.7.2.4 Display update. In Exercise 6, the digits are written only when the mode changes. `display_mode()` now also remembers the value on road H in `shown_value`; when B2 or a rejected B3 changes `temp_value`, only the output buffer is updated with `set7SegH()`, and `scan7Seg()` shows the new digits.

Table 20: Functions added or changed in `input_processing.c`

Function	Change
`increase_temp()` (new)	B2: `temp_value` + 1, 99 → 1
`save_red()` (new)	B3: save T_red and adjust T_green, or reject
`fsm_for_input_processing()`	Takes the B2 and B3 events; uses them in Mode 2
`display_mode()`	Refreshes road H when `temp_value` changes

Program 1.20: input_processing.c (updated)

```c
#include "main.h"
#include "input_reading.h"
#include "input_processing.h"
#include "software_timer.h"
#include "led_display.h"

#define AUTO_REPEAT_MS 500
#define BLINK_MS       250     /* toggle every 250 ms = 2 Hz */

/* ---------- FSM A: one copy per button (unchanged from Exercise 5) ---------- */
enum ButtonState {
    BUTTON_RELEASED,
    BUTTON_PRESSED,
    BUTTON_PRESSED_MORE_THAN_1_SECOND
};

static enum ButtonState buttonState[NO_OF_BUTTONS];
static uint8_t pressEvent[NO_OF_BUTTONS];
static uint8_t repeatEvent[NO_OF_BUTTONS];

static void fsm_button(uint8_t i)
{
    switch (buttonState[i]) {
    case BUTTON_RELEASED:
        if (is_button_pressed(i)) {
            buttonState[i] = BUTTON_PRESSED;
            pressEvent[i] = 1;
        }
        break;

    case BUTTON_PRESSED:
        if (!is_button_pressed(i)) {
            buttonState[i] = BUTTON_RELEASED;
        } else if (is_button_pressed_1s(i)) {
            buttonState[i] = BUTTON_PRESSED_MORE_THAN_1_SECOND;
            setTimer(TIMER_BTN0_REPEAT + i, AUTO_REPEAT_MS);
        }
```

```
38          break;
39
40      case BUTTON_PRESSED_MORE_THAN_1_SECOND:
41          if (!is_button_pressed(i)) {
42              buttonState[i] = BUTTON_RELEASED;
43          } else if (isTimerExpired(TIMER_BTN0_REPEAT + i)) {
44              repeatEvent[i] = 1;
45              setTimer(TIMER_BTN0_REPEAT + i, AUTO_REPEAT_MS);
46          }
47          break;
48      }
49  }
50
51  static uint8_t take(uint8_t *event)
52  {
53      uint8_t value = *event;
54      *event = 0;
55      return value;
56  }
57
58  /* ---------- Durations (seconds) and the edited value ---------- */
59  static uint8_t time_red   = 5;
60  static uint8_t time_amber = 2;
61  static uint8_t time_green = 3;
62  static uint8_t temp_value = 0;   /* value shown/edited in Modes 2-4 */
63
64  /* Copy the duration of the selected colour into temp_value */
65  static void load_temp(void)
66  {
67      if (get_mode() == MODE_RED)   temp_value = time_red;
68      if (get_mode() == MODE_AMBER) temp_value = time_amber;
69      if (get_mode() == MODE_GREEN) temp_value = time_green;
70  }
71
72  /* Exercise 7: B2 adds 1 to the edited value, 99 -> 1 */
73  static void increase_temp(void)
74  {
75      temp_value++;
76      if (temp_value > 99) {
77          temp_value = 1;
78      }
79  }
80
81  /* Exercise 7: B3 saves the red duration.
82     Rule: T_red = T_green + T_amber, so green is adjusted
83     and must stay >= 1 (red must be greater than amber). */
84  static void save_red(void)
85  {
86      if (temp_value > time_amber) {
87          time_red   = temp_value;
88          time_green = time_red - time_amber;
89      } else {
90          temp_value = time_red;    /* invalid: show the saved value again */
91      }
92  }
93
```

```c
/* ---------- FSM B: mode selection ---------- */
static uint8_t mode = MODE_NORMAL;

void fsm_for_input_processing(void)
{
    for (uint8_t i = 0; i < NO_OF_BUTTONS; i++) {
        fsm_button(i);
    }

    /* B1: one press = next mode, 1 -> 2 -> 3 -> 4 -> 1 */
    if (take(&pressEvent[BUTTON_MODE])) {
        mode++;
        if (mode > MODE_GREEN) {
            mode = MODE_NORMAL;
        }
        load_temp();              /* Exercise 6: value of the new mode */
    }
    repeatEvent[BUTTON_MODE] = 0;

    /* Exercise 7: read B2 and B3 in every mode, so no old event is
       left over when an edit mode is entered */
    uint8_t inc = take(&pressEvent[BUTTON_INC]);
    inc |= take(&repeatEvent[BUTTON_INC]);    /* held > 1 s: +1 every 500 ms */
    uint8_t set = take(&pressEvent[BUTTON_SET]);
    repeatEvent[BUTTON_SET] = 0;              /* holding B3 saves only once */

    if (mode == MODE_RED) {
        if (inc) increase_temp();
        if (set) save_red();
    }
}

uint8_t get_mode(void)
{
    return mode;
}

/* ---------- Exercise 6: display the mode ---------- */
static uint8_t shown_mode  = 0;  /* 0 = nothing shown yet */
static uint8_t shown_value = 0;  /* value now on road H (Exercise 7) */
static uint8_t blink_on    = 0;  /* 1 = blinking LEDs are on now */

/* Colour that blinks in the current mode */
static uint8_t mode_colour(void)
{
    if (mode == MODE_RED)   return LIGHT_RED;
    if (mode == MODE_AMBER) return LIGHT_AMBER;
    if (mode == MODE_GREEN) return LIGHT_GREEN;
    return LIGHT_OFF;             /* Mode 1: traffic FSM added in Exercise 10 */
}

static void update_lights(void)
{
    uint8_t light = blink_on ? mode_colour() : LIGHT_OFF;
    setLightH(light);
    setLightV(light);
```

```
150 }
151
152 void display_mode(void)
153 {
154     if (mode != shown_mode) {
155         /* A new mode was entered: update everything once */
156         shown_mode = mode;
157
158         if (mode == MODE_NORMAL) {
159             set7SegH(SEG_BLANK);     /* no value in Mode 1 (yet) */
160         } else {
161             set7SegH(temp_value);    /* road H: the value  */
162         }
163         set7SegV(mode);              /* road V: the mode, e.g. 02 */
164         shown_value = temp_value;
165
166         blink_on = 1;                /* start with the LEDs on */
167         setTimer(TIMER_BLINK, BLINK_MS);
168         update_lights();
169     }
170     else if (mode != MODE_NORMAL && isTimerExpired(TIMER_BLINK)) {
171         /* Same mode: toggle the colour every 250 ms (2 Hz) */
172         blink_on = !blink_on;
173         setTimer(TIMER_BLINK, BLINK_MS);
174         update_lights();
175     }
176
177     /* Exercise 7: the value was changed by B2 (or restored by B3) */
178     if (mode != MODE_NORMAL && temp_value != shown_value) {
179         shown_value = temp_value;
180         set7SegH(temp_value);
181     }
182 }
```

1.7.3 Result

Table 21: Test results of Exercise 7 (defaults: red 5, amber 2, green 3)

Test	Result
B2 and B3 in Mode 1, then B1	Mode 2 shows 05: no stale event
B2 three times	06, 07, 08
B1 without B3, back to Mode 2	05: the draft is discarded
B2 three times, then B3	T_red = 8
Mode 3 and Mode 4	Amber 02; green adjusted to 06
B2 or B3 in Modes 3 and 4	No change (Exercises 8 and 9)
Hold B2 for 3 s	08 $\rightarrow$ 12: +1, then +1 every 500 ms after 1 s
Hold B2 past 99	98, 99, 01
B3 with value 02 ($\leq$ T_amber)	Rejected; 08 is shown again

A software simulation with a mocked HAL gives the results of Table 21 for interrupt periods of 1, 10, and 100 ms, and the red LEDs keep blinking at 2 Hz while the value is edited.

1.8 Exercise 8: Adding Code for Increasing Time Duration Value for the Amber LEDs

1.8.1 Requirement

Your tasks in this exercise are:

- to use the second button to increase the time duration value of the amber LEDs
- to use the third button to set the value for the amber LEDs.

1.8.2 Implementation

This exercise implements the self-loops of Mode 3 in the mode FSM (Figure 2). Only `input_processing.c` changes; all other files are the same as in Exercise 7. Most of the code of Exercise 7 is reused without change, because it works on the shared draft `temp_value`: `increase_temp()` (+1, 99 $\rightarrow$ 1), the reading of the B2 and B3 events, the discarding of an unsaved draft by `load_temp()`, and the display refresh in `display_mode()`.

1.8.2.1 Increasing the value. The B2 condition is extended to Mode 3, so B2 now increases `temp_value` in Modes 2 and 3. B3 calls the save function of the current mode.

1.8.2.2 Setting the value. As in Exercise 7, the rule $T_{\text{red}} = T_{\text{green}} + T_{\text{amber}}$ is kept. When B3 is pressed in Mode 3, `save_amber()` keeps T_red and:

- saves `temp_value` as T_amber and sets T_green = T_red − T_amber, if T_amber < T_red;
- otherwise rejects the value, because T_green would be 0 or negative, and restores the saved amber duration on the display.

Editing red or amber therefore adjusts green; editing green is handled in Exercise 9.

Table 22: Changes in `input_processing.c`

Function	Change
`save_amber()` (new)	B3: save T_amber and adjust T_green, or reject
`fsm_for_input_processing()`	B2 also works in Mode 3; B3 calls the save function of the current mode

Program 1.21: input_processing.c (updated)

```c
#include "main.h"
#include "input_reading.h"
#include "input_processing.h"
#include "software_timer.h"
#include "led_display.h"

#define AUTO_REPEAT_MS 500
#define BLINK_MS       250     /* toggle every 250 ms = 2 Hz */

/* ---------- FSM A: one copy per button (unchanged from Exercise 5) ---------- */
enum ButtonState {
    BUTTON_RELEASED,
    BUTTON_PRESSED,
    BUTTON_PRESSED_MORE_THAN_1_SECOND
};
```

```
16
17  static enum ButtonState buttonState[NO_OF_BUTTONS];
18  static uint8_t pressEvent[NO_OF_BUTTONS];
19  static uint8_t repeatEvent[NO_OF_BUTTONS];
20
21  static void fsm_button(uint8_t i)
22  {
23      switch (buttonState[i]) {
24      case BUTTON_RELEASED:
25          if (is_button_pressed(i)) {
26              buttonState[i] = BUTTON_PRESSED;
27              pressEvent[i] = 1;
28          }
29          break;
30
31      case BUTTON_PRESSED:
32          if (!is_button_pressed(i)) {
33              buttonState[i] = BUTTON_RELEASED;
34          } else if (is_button_pressed_1s(i)) {
35              buttonState[i] = BUTTON_PRESSED_MORE_THAN_1_SECOND;
36              setTimer(TIMER_BTN0_REPEAT + i, AUTO_REPEAT_MS);
37          }
38          break;
39
40      case BUTTON_PRESSED_MORE_THAN_1_SECOND:
41          if (!is_button_pressed(i)) {
42              buttonState[i] = BUTTON_RELEASED;
43          } else if (isTimerExpired(TIMER_BTN0_REPEAT + i)) {
44              repeatEvent[i] = 1;
45              setTimer(TIMER_BTN0_REPEAT + i, AUTO_REPEAT_MS);
46          }
47          break;
48      }
49  }
50
51  static uint8_t take(uint8_t *event)
52  {
53      uint8_t value = *event;
54      *event = 0;
55      return value;
56  }
57
58  /* ---------- Durations (seconds) and the edited value ---------- */
59  static uint8_t time_red   = 5;
60  static uint8_t time_amber = 2;
61  static uint8_t time_green = 3;
62  static uint8_t temp_value = 0;   /* value shown/edited in Modes 2-4 */
63
64  /* Copy the duration of the selected colour into temp_value */
65  static void load_temp(void)
66  {
67      if (get_mode() == MODE_RED)   temp_value = time_red;
68      if (get_mode() == MODE_AMBER) temp_value = time_amber;
69      if (get_mode() == MODE_GREEN) temp_value = time_green;
70  }
71
```

```c
/* Exercise 7: B2 adds 1 to the edited value, 99 -> 1 */
static void increase_temp(void)
{
    temp_value++;
    if (temp_value > 99) {
        temp_value = 1;
    }
}

/* Exercise 7: B3 saves the red duration.
   Rule: T_red = T_green + T_amber, so green is adjusted
   and must stay >= 1 (red must be greater than amber). */
static void save_red(void)
{
    if (temp_value > time_amber) {
        time_red   = temp_value;
        time_green = time_red - time_amber;
    } else {
        temp_value = time_red;    /* invalid: show the saved value again */
    }
}

/* Exercise 8: B3 saves the amber duration.
   Red is kept, green is adjusted: T_green = T_red - T_amber,
   so amber must be smaller than red (green >= 1). */
static void save_amber(void)
{
    if (temp_value < time_red) {
        time_amber = temp_value;
        time_green = time_red - time_amber;
    } else {
        temp_value = time_amber;  /* invalid: show the saved value again */
    }
}

/* ---------- FSM B: mode selection ---------- */
static uint8_t mode = MODE_NORMAL;

void fsm_for_input_processing(void)
{
    for (uint8_t i = 0; i < NO_OF_BUTTONS; i++) {
        fsm_button(i);
    }

    /* B1: one press = next mode, 1 -> 2 -> 3 -> 4 -> 1 */
    if (take(&pressEvent[BUTTON_MODE])) {
        mode++;
        if (mode > MODE_GREEN) {
            mode = MODE_NORMAL;
        }
        load_temp();              /* Exercise 6: value of the new mode */
    }
    repeatEvent[BUTTON_MODE] = 0;

    /* Exercise 7: read B2 and B3 in every mode, so no old event is
       left over when an edit mode is entered */
```

```
128     uint8_t inc = take(&pressEvent[BUTTON_INC]);
129     inc |= take(&repeatEvent[BUTTON_INC]);    /* held > 1 s: +1 every 500 ms */
130     uint8_t set = take(&pressEvent[BUTTON_SET]);
131     repeatEvent[BUTTON_SET] = 0;              /* holding B3 saves only once */
132
133     /* B2: increase the draft in the edit modes done so far */
134     if (inc && (mode == MODE_RED || mode == MODE_AMBER)) {
135         increase_temp();
136     }
137
138     /* B3: save the draft into the duration of the selected colour */
139     if (set) {
140         if (mode == MODE_RED)   save_red();      /* Exercise 7 */
141         if (mode == MODE_AMBER) save_amber();    /* Exercise 8 */
142     }
143 }
144
145 uint8_t get_mode(void)
146 {
147     return mode;
148 }
149
150 /* ---------- Exercise 6: display the mode ---------- */
151 static uint8_t shown_mode  = 0;  /* 0 = nothing shown yet */
152 static uint8_t shown_value = 0;  /* value now on road H (Exercise 7) */
153 static uint8_t blink_on    = 0;  /* 1 = blinking LEDs are on now */
154
155 /* Colour that blinks in the current mode */
156 static uint8_t mode_colour(void)
157 {
158     if (mode == MODE_RED)   return LIGHT_RED;
159     if (mode == MODE_AMBER) return LIGHT_AMBER;
160     if (mode == MODE_GREEN) return LIGHT_GREEN;
161     return LIGHT_OFF;             /* Mode 1: traffic FSM added in Exercise 10 */
162 }
163
164 static void update_lights(void)
165 {
166     uint8_t light = blink_on ? mode_colour() : LIGHT_OFF;
167     setLightH(light);
168     setLightV(light);
169 }
170
171 void display_mode(void)
172 {
173     if (mode != shown_mode) {
174         /* A new mode was entered: update everything once */
175         shown_mode = mode;
176
177         if (mode == MODE_NORMAL) {
178             set7SegH(SEG_BLANK);     /* no value in Mode 1 (yet) */
179         } else {
180             set7SegH(temp_value);    /* road H: the value  */
181         }
182         set7SegV(mode);              /* road V: the mode, e.g. 02 */
183         shown_value = temp_value;
```

```
184
185         blink_on = 1;                /* start with the LEDs on */
186         setTimer(TIMER_BLINK, BLINK_MS);
187         update_lights();
188     }
189     else if (mode != MODE_NORMAL && isTimerExpired(TIMER_BLINK)) {
190         /* Same mode: toggle the colour every 250 ms (2 Hz) */
191         blink_on = !blink_on;
192         setTimer(TIMER_BLINK, BLINK_MS);
193         update_lights();
194     }
195
196     /* Exercise 7: the value was changed by B2 (or restored by B3) */
197     if (mode != MODE_NORMAL && temp_value != shown_value) {
198         shown_value = temp_value;
199         set7SegH(temp_value);
200     }
201 }
```

1.8.3 Result

Table 23: Test results of Exercise 8 (defaults: red 5, amber 2, green 3)

Test	Result
B1 twice	Mode 3: amber LEDs blink, H = 02, V = 03
B2 once	03
B1 without B3, back to Mode 3	02: the draft is discarded
B2 once, then B3	T_amber = 3
Mode 4, then Mode 2	Green adjusted to 02; red unchanged at 05
B2 to 05 ($\geq$ T_red), then B3	Rejected; 03 is shown again
B2 to 04, then B3	Saved; Mode 4 shows green 01
Hold B2	+1 every 500 ms after 1 s; 99 $\rightarrow$ 01
B2 or B3 in Mode 4	No change (Exercise 9)
Red editing of Exercise 7	Still works

A software simulation with a mocked HAL confirms these results for interrupt periods of 1, 10, and 100 ms, and the amber LEDs keep blinking at 2 Hz while the value is edited.

1.9 Exercise 9: Adding Code for Increasing Time Duration Value for the Green LEDs

1.9.1 Requirement

Your tasks in this exercise are:

- to use the second button to increase the time duration value of the green LEDs
- to use the third button to set the value for the green LEDs.

1.9.2 Implementation

This exercise implements the self-loops of Mode 4 in the mode FSM (Figure 2) and completes the three edit modes. Only `input_processing.c` changes; all other files are the same as in

Exercise 8. As before, `increase_temp()`, the reading of the B2 and B3 events, the discarding of an unsaved draft, and the display refresh are reused without change.

1.9.2.1 Increasing the value. The B2 condition is extended to Mode 4, so B2 now increases `temp_value` in all three edit modes. B3 calls `save_green()` in Mode 4.

1.9.2.2 Setting the value. When B3 is pressed in Mode 4, `save_green()` keeps T_amber and:

- saves `temp_value` as T_green and sets T_red = T_green + T_amber, if this sum is at most 99;
- otherwise rejects the value, because T_red would not fit in two digits, and restores the saved green duration on the display.

No lower check is needed, since `temp_value` is always at least 1. Table 24 summarises how the three save functions keep $T_{\text{red}} = T_{\text{green}} + T_{\text{amber}}$; amber is never changed automatically.

Table 24: Keeping the duration rule in Exercises 7–9

Mode	Saved	Adjusted	Rejected if
2	T_red	T_green = T_red − T_amber	T_red ≤ T_amber
3	T_amber	T_green = T_red − T_amber	T_amber ≥ T_red
4	T_green	T_red = T_green + T_amber	T_green + T_amber > 99

Table 25: Changes in `input_processing.c`

Function	Change
`save_green()` (new)	B3: save T_green and adjust T_red, or reject
`fsm_for_input_processing()`	B2 also works in Mode 4; B3 calls `save_green()` in Mode 4

Program 1.22: input_processing.c (updated)

```c
#include "main.h"
#include "input_reading.h"
#include "input_processing.h"
#include "software_timer.h"
#include "led_display.h"

#define AUTO_REPEAT_MS 500
#define BLINK_MS       250     /* toggle every 250 ms = 2 Hz */

/* ---------- FSM A: one copy per button (unchanged from Exercise 5) ---------- */
enum ButtonState {
    BUTTON_RELEASED,
    BUTTON_PRESSED,
    BUTTON_PRESSED_MORE_THAN_1_SECOND
};

static enum ButtonState buttonState[NO_OF_BUTTONS];
static uint8_t pressEvent[NO_OF_BUTTONS];
static uint8_t repeatEvent[NO_OF_BUTTONS];

```

```c
static void fsm_button(uint8_t i)
{
    switch (buttonState[i]) {
    case BUTTON_RELEASED:
        if (is_button_pressed(i)) {
            buttonState[i] = BUTTON_PRESSED;
            pressEvent[i] = 1;
        }
        break;

    case BUTTON_PRESSED:
        if (!is_button_pressed(i)) {
            buttonState[i] = BUTTON_RELEASED;
        } else if (is_button_pressed_1s(i)) {
            buttonState[i] = BUTTON_PRESSED_MORE_THAN_1_SECOND;
            setTimer(TIMER_BTN0_REPEAT + i, AUTO_REPEAT_MS);
        }
        break;

    case BUTTON_PRESSED_MORE_THAN_1_SECOND:
        if (!is_button_pressed(i)) {
            buttonState[i] = BUTTON_RELEASED;
        } else if (isTimerExpired(TIMER_BTN0_REPEAT + i)) {
            repeatEvent[i] = 1;
            setTimer(TIMER_BTN0_REPEAT + i, AUTO_REPEAT_MS);
        }
        break;
    }
}

static uint8_t take(uint8_t *event)
{
    uint8_t value = *event;
    *event = 0;
    return value;
}

/* ---------- Durations (seconds) and the edited value ---------- */
static uint8_t time_red   = 5;
static uint8_t time_amber = 2;
static uint8_t time_green = 3;
static uint8_t temp_value = 0;   /* value shown/edited in Modes 2-4 */

/* Copy the duration of the selected colour into temp_value */
static void load_temp(void)
{
    if (get_mode() == MODE_RED)   temp_value = time_red;
    if (get_mode() == MODE_AMBER) temp_value = time_amber;
    if (get_mode() == MODE_GREEN) temp_value = time_green;
}

/* Exercise 7: B2 adds 1 to the edited value, 99 -> 1 */
static void increase_temp(void)
{
    temp_value++;
    if (temp_value > 99) {
```

```
77          temp_value = 1;
78      }
79  }
80
81  /* Exercise 7: B3 saves the red duration.
82     Rule: T_red = T_green + T_amber, so green is adjusted
83     and must stay >= 1 (red must be greater than amber). */
84  static void save_red(void)
85  {
86      if (temp_value > time_amber) {
87          time_red   = temp_value;
88          time_green = time_red - time_amber;
89      } else {
90          temp_value = time_red;    /* invalid: show the saved value again */
91      }
92  }
93
94  /* Exercise 8: B3 saves the amber duration.
95     Red is kept, green is adjusted: T_green = T_red - T_amber,
96     so amber must be smaller than red (green >= 1). */
97  static void save_amber(void)
98  {
99      if (temp_value < time_red) {
100         time_amber = temp_value;
101         time_green = time_red - time_amber;
102     } else {
103         temp_value = time_amber;  /* invalid: show the saved value again */
104     }
105 }
106
107 /* Exercise 9: B3 saves the green duration.
108    Amber is kept, red is adjusted: T_red = T_green + T_amber,
109    so red must still fit in two digits (red <= 99). */
110 static void save_green(void)
111 {
112     if (temp_value + time_amber <= 99) {
113         time_green = temp_value;
114         time_red   = time_green + time_amber;
115     } else {
116         temp_value = time_green;  /* invalid: show the saved value again */
117     }
118 }
119
120 /* ---------- FSM B: mode selection ---------- */
121 static uint8_t mode = MODE_NORMAL;
122
123 void fsm_for_input_processing(void)
124 {
125     for (uint8_t i = 0; i < NO_OF_BUTTONS; i++) {
126         fsm_button(i);
127     }
128
129     /* B1: one press = next mode, 1 -> 2 -> 3 -> 4 -> 1 */
130     if (take(&pressEvent[BUTTON_MODE])) {
131         mode++;
132         if (mode > MODE_GREEN) {
```

```
133             mode = MODE_NORMAL;
134         }
135         load_temp();              /* Exercise 6: value of the new mode */
136     }
137     repeatEvent[BUTTON_MODE] = 0;
138 
139     /* Exercise 7: read B2 and B3 in every mode, so no old event is
140        left over when an edit mode is entered */
141     uint8_t inc = take(&pressEvent[BUTTON_INC]);
142     inc |= take(&repeatEvent[BUTTON_INC]);    /* held > 1 s: +1 every 500 ms */
143     uint8_t set = take(&pressEvent[BUTTON_SET]);
144     repeatEvent[BUTTON_SET] = 0;              /* holding B3 saves only once */
145 
146     /* B2: increase the draft in the three edit modes */
147     if (inc && (mode == MODE_RED || mode == MODE_AMBER || mode == MODE_GREEN)) {
148         increase_temp();
149     }
150 
151     /* B3: save the draft into the duration of the selected colour */
152     if (set) {
153         if (mode == MODE_RED)   save_red();      /* Exercise 7 */
154         if (mode == MODE_AMBER) save_amber();    /* Exercise 8 */
155         if (mode == MODE_GREEN) save_green();    /* Exercise 9 */
156     }
157 }
158 
159 uint8_t get_mode(void)
160 {
161     return mode;
162 }
163 
164 /* ---------- Exercise 6: display the mode ---------- */
165 static uint8_t shown_mode  = 0;  /* 0 = nothing shown yet */
166 static uint8_t shown_value = 0;  /* value now on road H (Exercise 7) */
167 static uint8_t blink_on    = 0;  /* 1 = blinking LEDs are on now */
168 
169 /* Colour that blinks in the current mode */
170 static uint8_t mode_colour(void)
171 {
172     if (mode == MODE_RED)   return LIGHT_RED;
173     if (mode == MODE_AMBER) return LIGHT_AMBER;
174     if (mode == MODE_GREEN) return LIGHT_GREEN;
175     return LIGHT_OFF;             /* Mode 1: traffic FSM added in Exercise 10 */
176 }
177 
178 static void update_lights(void)
179 {
180     uint8_t light = blink_on ? mode_colour() : LIGHT_OFF;
181     setLightH(light);
182     setLightV(light);
183 }
184 
185 void display_mode(void)
186 {
187     if (mode != shown_mode) {
188         /* A new mode was entered: update everything once */
```

```
189             shown_mode = mode;
190
191             if (mode == MODE_NORMAL) {
192                 set7SegH(SEG_BLANK);     /* no value in Mode 1 (yet) */
193             } else {
194                 set7SegH(temp_value);    /* road H: the value  */
195             }
196             set7SegV(mode);              /* road V: the mode, e.g. 02 */
197             shown_value = temp_value;
198
199             blink_on = 1;                /* start with the LEDs on */
200             setTimer(TIMER_BLINK, BLINK_MS);
201             update_lights();
202         }
203         else if (mode != MODE_NORMAL && isTimerExpired(TIMER_BLINK)) {
204             /* Same mode: toggle the colour every 250 ms (2 Hz) */
205             blink_on = !blink_on;
206             setTimer(TIMER_BLINK, BLINK_MS);
207             update_lights();
208         }
209
210         /* Exercise 7: the value was changed by B2 (or restored by B3) */
211         if (mode != MODE_NORMAL && temp_value != shown_value) {
212             shown_value = temp_value;
213             set7SegH(temp_value);
214         }
215     }
```

1.9.3 Result

Table 26: Test results of Exercise 9 (defaults: red 5, amber 2, green 3)

Test	Result
B1 three times	Mode 4: green LEDs blink, H = 03, V = 04
B2 once	04
B1 without B3, back to Mode 4	03: the draft is discarded
B2 once, then B3	T_green = 4
Mode 2, then Mode 3	Red adjusted to 06; amber unchanged at 02
Hold B2 to 98, then B3	Rejected (98 + 2 > 99); 04 is shown again
Hold B2 to 97, then B3	Saved; Mode 2 shows red 99
B2 past 99	97, 98, 99, 01
Red and amber editing (Exercises 7, 8)	Still work

All tests pass in Proteus, and the green LEDs keep blinking at 2 Hz while the value is edited. A software simulation with a mocked HAL gives the same results for interrupt periods of 1, 10, and 100 ms.

1.10 Exercise 10: To Finish the Project

1.10.1 Requirement

Your tasks in this exercise are:

- To integrate all the previous tasks to one final project
- To create a video to show all features in the specification (no need)
- To add a report to describe your solution for each exercise.
- To submit your report and code on the BKeL

1.10.2 Implementation

After Exercise 9, all four modes can be selected and edited, but Mode 1 does not yet run the traffic light. This exercise adds the traffic-light FSM of Figure 3 and integrates it with the modules of Exercises 4–9.

1.10.2.1 Traffic-light FSM. The new module `traffic_light.c` implements FSM C with the four states of Table 8.

- `enter_state()` performs the entry actions of a state: it loads `count` with T_green (S0, S2) or T_amber (S1, S3) and sets the lights of both roads.
- `traffic_run()` is called in every pass of the main loop while in Mode 1. Each time the 1-s timer `TIMER_SECOND` expires, it decreases `count`; when `count` reaches 0, it enters the next state, and S3 returns to S0. The function never blocks.
- `traffic_init()` restarts the FSM from S0. It is called whenever Mode 1 is entered, which implements the transition `B1 pressed / restart traffic FSM` of the mode FSM.

1.10.2.2 Countdown display. Each road shows the remaining time of its own light. The road that is green or amber shows `count`. The red road waits for the green and then the amber of the other road, so while the other road is green it shows `count` + T_amber, and while the other road is amber it shows `count`. At the start of S0, road H therefore shows T_green + T_amber = T_red. Because Exercises 7–9 keep $T_{\text{red}} = T_{\text{green}} + T_{\text{amber}}$ (Table 24), the red countdown always matches the time for which the red light is on, and the FSM needs only T_green and T_amber.

1.10.2.3 Integration. The durations stay private to `input_processing.c`; the traffic light reads them through `get_time_amber()` and `get_time_green()`. `display_mode()` now runs the traffic light in Mode 1 and the edit display of Exercise 6 in Modes 2–4. The traffic light writes only to the output buffer and the lights, so `main.c` is unchanged from Exercise 6. Table 27 lists the modules of the final project.

Table 27: Modules of the final project

File	Function	From
`software_timer`	Software timers in milliseconds	Exercise 4
`input_reading`	Debouncing; 1-s hold detection	Exercise 5
`input_processing`	Button FSM, mode FSM, editing, display per mode	Exercises 5–10
`led_display`	Traffic LEDs; seven-segment buffer and scanning	Exercises 5, 6
`traffic_light` (new)	Traffic-light FSM and countdown in Mode 1	Exercise 10
`main.c`	Starts TIM2; runs the modules in the main loop	Exercise 6

Table 28: Files changed or added in Exercise 10

File	Change
`traffic_light.h/.c` (new)	Traffic-light FSM
`software_timer.h`	Adds `TIMER_SECOND`
`input_processing.h/.c`	Adds the two getters; runs the traffic light in Mode 1
`CMakeLists.txt`	Adds `traffic_light.c` to the build

Program 1.23: traffic_light.h (new)

```
#ifndef INC_TRAFFIC_LIGHT_H_
#define INC_TRAFFIC_LIGHT_H_

void traffic_init(void);   /* start the traffic light from S0     */
void traffic_run(void);    /* call in the main loop while in Mode 1 */

#endif /* INC_TRAFFIC_LIGHT_H_ */
```

Program 1.24: traffic_light.c (new)

```
#include "main.h"
#include "traffic_light.h"
#include "input_processing.h"
#include "led_display.h"
#include "software_timer.h"

#define ONE_SECOND_MS 1000

/* ---------- FSM C: traffic light (Exercise 1) ---------- */
enum TrafficState {
    S0_H_RED_V_GREEN,
    S1_H_RED_V_AMBER,
    S2_H_GREEN_V_RED,
    S3_H_AMBER_V_RED
};

static enum TrafficState state = S0_H_RED_V_GREEN;
static uint8_t count = 0;   /* seconds left in the current state */

/* Each road shows the time left of its own light.
   A red road waits for the other road's green and amber,
   so during green its red countdown is count + T_amber. */
static void show_countdown(void)
{
    switch (state) {
    case S0_H_RED_V_GREEN:
        set7SegH(count + get_time_amber());   /* red   */
        set7SegV(count);                      /* green */
        break;
    case S1_H_RED_V_AMBER:
        set7SegH(count);                      /* red   */
        set7SegV(count);                      /* amber */
        break;
    case S2_H_GREEN_V_RED:
```

```
35          set7SegH(count);                        /* green */
36          set7SegV(count + get_time_amber());     /* red   */
37          break;
38      case S3_H_AMBER_V_RED:
39          set7SegH(count);                        /* amber */
40          set7SegV(count);                        /* red   */
41          break;
42      }
43  }
44
45  /* Entry action of a state: load count and set the lights */
46  static void enter_state(enum TrafficState next)
47  {
48      state = next;
49
50      switch (state) {
51      case S0_H_RED_V_GREEN:
52          count = get_time_green();
53          setLightH(LIGHT_RED);
54          setLightV(LIGHT_GREEN);
55          break;
56      case S1_H_RED_V_AMBER:
57          count = get_time_amber();
58          setLightH(LIGHT_RED);
59          setLightV(LIGHT_AMBER);
60          break;
61      case S2_H_GREEN_V_RED:
62          count = get_time_green();
63          setLightH(LIGHT_GREEN);
64          setLightV(LIGHT_RED);
65          break;
66      case S3_H_AMBER_V_RED:
67          count = get_time_amber();
68          setLightH(LIGHT_AMBER);
69          setLightV(LIGHT_RED);
70          break;
71      }
72
73      show_countdown();
74  }
75
76  void traffic_init(void)
77  {
78      enter_state(S0_H_RED_V_GREEN);
79      setTimer(TIMER_SECOND, ONE_SECOND_MS);
80  }
81
82  void traffic_run(void)
83  {
84      if (!isTimerExpired(TIMER_SECOND)) {
85          return;                             /* wait for the next second */
86      }
87      setTimer(TIMER_SECOND, ONE_SECOND_MS);
88
89      count--;                                /* count > 0 / count-- (1 s) */
90
```

```
91      if (count > 0) {
92          show_countdown();
93          return;
94      }
95
96      /* count = 0: go to the next state */
97      switch (state) {
98      case S0_H_RED_V_GREEN: enter_state(S1_H_RED_V_AMBER); break;
99      case S1_H_RED_V_AMBER: enter_state(S2_H_GREEN_V_RED); break;
100     case S2_H_GREEN_V_RED: enter_state(S3_H_AMBER_V_RED); break;
101     case S3_H_AMBER_V_RED: enter_state(S0_H_RED_V_GREEN); break;
102     }
103 }
```

Program 1.25: software_timer.h (updated)

```
1  #ifndef INC_SOFTWARE_TIMER_H_
2  #define INC_SOFTWARE_TIMER_H_
3
4  #include <stdint.h>
5
6  /* TIM2 interrupt period in ms. Must match PSC/ARR in CubeMX. */
7  #ifndef TIMER_CYCLE
8  #define TIMER_CYCLE 10
9  #endif
10
11 /* One index per software timer */
12 enum {
13     TIMER_BLINK = 0,      /* 2 Hz blinking in Modes 2-4            */
14     TIMER_BTN0_REPEAT,    /* auto-repeat of button 0 when held     */
15     TIMER_BTN1_REPEAT,    /* auto-repeat of button 1 when held     */
16     TIMER_BTN2_REPEAT,    /* auto-repeat of button 2 when held     */
17     TIMER_SCAN,           /* Exercise 6: 7-segment scanning        */
18     TIMER_SECOND,         /* Exercise 10: 1 s traffic countdown    */
19     NUM_OF_TIMERS
20 };
21
22 void    setTimer(uint8_t index, uint32_t duration_ms);
23 uint8_t isTimerExpired(uint8_t index);
24 void    timerRun(void);
25
26 #endif /* INC_SOFTWARE_TIMER_H_ */
```

Program 1.26: input_processing.h (updated)

```
1  #ifndef INC_INPUT_PROCESSING_H_
2  #define INC_INPUT_PROCESSING_H_
3
4  #include <stdint.h>
5
6  /* Modes of the application */
```

```
enum {
    MODE_NORMAL = 1,    /* Mode 1 */
    MODE_RED,           /* Mode 2 */
    MODE_AMBER,         /* Mode 3 */
    MODE_GREEN          /* Mode 4 */
};

void    fsm_for_input_processing(void);
void    display_mode(void);          /* Exercise 6 */
uint8_t get_mode(void);
uint8_t get_time_amber(void);        /* Exercise 10 */
uint8_t get_time_green(void);        /* Exercise 10 */

#endif /* INC_INPUT_PROCESSING_H_ */
```

Program 1.27: input_processing.c (final)

```
#include "main.h"
#include "input_reading.h"
#include "input_processing.h"
#include "software_timer.h"
#include "led_display.h"
#include "traffic_light.h"

#define AUTO_REPEAT_MS 500
#define BLINK_MS       250     /* toggle every 250 ms = 2 Hz */

/* ---------- FSM A: one copy per button (unchanged from Exercise 5) ---------- */
enum ButtonState {
    BUTTON_RELEASED,
    BUTTON_PRESSED,
    BUTTON_PRESSED_MORE_THAN_1_SECOND
};

static enum ButtonState buttonState[NO_OF_BUTTONS];
static uint8_t pressEvent[NO_OF_BUTTONS];
static uint8_t repeatEvent[NO_OF_BUTTONS];

static void fsm_button(uint8_t i)
{
    switch (buttonState[i]) {
    case BUTTON_RELEASED:
        if (is_button_pressed(i)) {
            buttonState[i] = BUTTON_PRESSED;
            pressEvent[i] = 1;
        }
        break;

    case BUTTON_PRESSED:
        if (!is_button_pressed(i)) {
            buttonState[i] = BUTTON_RELEASED;
        } else if (is_button_pressed_1s(i)) {
            buttonState[i] = BUTTON_PRESSED_MORE_THAN_1_SECOND;
            setTimer(TIMER_BTN0_REPEAT + i, AUTO_REPEAT_MS);
```

```
        }
        break;

    case BUTTON_PRESSED_MORE_THAN_1_SECOND:
        if (!is_button_pressed(i)) {
            buttonState[i] = BUTTON_RELEASED;
        } else if (isTimerExpired(TIMER_BTN0_REPEAT + i)) {
            repeatEvent[i] = 1;
            setTimer(TIMER_BTN0_REPEAT + i, AUTO_REPEAT_MS);
        }
        break;
    }
}

static uint8_t take(uint8_t *event)
{
    uint8_t value = *event;
    *event = 0;
    return value;
}

/* ---------- Durations (seconds) and the edited value ---------- */
static uint8_t time_red   = 5;
static uint8_t time_amber = 2;
static uint8_t time_green = 3;
static uint8_t temp_value = 0;   /* value shown/edited in Modes 2-4 */

/* Copy the duration of the selected colour into temp_value */
static void load_temp(void)
{
    if (get_mode() == MODE_RED)   temp_value = time_red;
    if (get_mode() == MODE_AMBER) temp_value = time_amber;
    if (get_mode() == MODE_GREEN) temp_value = time_green;
}

/* Exercise 7: B2 adds 1 to the edited value, 99 -> 1 */
static void increase_temp(void)
{
    temp_value++;
    if (temp_value > 99) {
        temp_value = 1;
    }
}

/* Exercise 7: B3 saves the red duration.
   Rule: T_red = T_green + T_amber, so green is adjusted
   and must stay >= 1 (red must be greater than amber). */
static void save_red(void)
{
    if (temp_value > time_amber) {
        time_red   = temp_value;
        time_green = time_red - time_amber;
    } else {
        temp_value = time_red;    /* invalid: show the saved value again */
    }
}
```

```
94
95  /* Exercise 8: B3 saves the amber duration.
96     Red is kept, green is adjusted: T_green = T_red - T_amber,
97     so amber must be smaller than red (green >= 1). */
98  static void save_amber(void)
99  {
100     if (temp_value < time_red) {
101         time_amber = temp_value;
102         time_green = time_red - time_amber;
103     } else {
104         temp_value = time_amber;  /* invalid: show the saved value again */
105     }
106 }
107
108 /* Exercise 9: B3 saves the green duration.
109    Amber is kept, red is adjusted: T_red = T_green + T_amber,
110    so red must still fit in two digits (red <= 99). */
111 static void save_green(void)
112 {
113     if (temp_value + time_amber <= 99) {
114         time_green = temp_value;
115         time_red   = time_green + time_amber;
116     } else {
117         temp_value = time_green;  /* invalid: show the saved value again */
118     }
119 }
120
121 /* ---------- FSM B: mode selection ---------- */
122 static uint8_t mode = MODE_NORMAL;
123
124 void fsm_for_input_processing(void)
125 {
126     for (uint8_t i = 0; i < NO_OF_BUTTONS; i++) {
127         fsm_button(i);
128     }
129
130     /* B1: one press = next mode, 1 -> 2 -> 3 -> 4 -> 1 */
131     if (take(&pressEvent[BUTTON_MODE])) {
132         mode++;
133         if (mode > MODE_GREEN) {
134             mode = MODE_NORMAL;
135         }
136         load_temp();              /* Exercise 6: value of the new mode */
137     }
138     repeatEvent[BUTTON_MODE] = 0;
139
140     /* Exercise 7: read B2 and B3 in every mode, so no old event is
141        left over when an edit mode is entered */
142     uint8_t inc = take(&pressEvent[BUTTON_INC]);
143     inc |= take(&repeatEvent[BUTTON_INC]);    /* held > 1 s: +1 every 500 ms */
144     uint8_t set = take(&pressEvent[BUTTON_SET]);
145     repeatEvent[BUTTON_SET] = 0;              /* holding B3 saves only once */
146
147     /* B2: increase the draft in the three edit modes */
148     if (inc && (mode == MODE_RED || mode == MODE_AMBER || mode == MODE_GREEN)) {
149         increase_temp();
```

```
150     }
151
152     /* B3: save the draft into the duration of the selected colour */
153     if (set) {
154         if (mode == MODE_RED)   save_red();      /* Exercise 7 */
155         if (mode == MODE_AMBER) save_amber();    /* Exercise 8 */
156         if (mode == MODE_GREEN) save_green();    /* Exercise 9 */
157     }
158 }
159
160 uint8_t get_mode(void)
161 {
162     return mode;
163 }
164
165 /* Exercise 10: saved durations used by the traffic light */
166 uint8_t get_time_amber(void)
167 {
168     return time_amber;
169 }
170
171 uint8_t get_time_green(void)
172 {
173     return time_green;
174 }
175
176 /* ---------- Exercise 6: display the mode ---------- */
177 static uint8_t shown_mode  = 0;  /* 0 = nothing shown yet */
178 static uint8_t shown_value = 0;  /* value now on road H (Exercise 7) */
179 static uint8_t blink_on    = 0;  /* 1 = blinking LEDs are on now */
180
181 /* Colour that blinks in the current mode */
182 static uint8_t mode_colour(void)
183 {
184     if (mode == MODE_RED)   return LIGHT_RED;
185     if (mode == MODE_AMBER) return LIGHT_AMBER;
186     if (mode == MODE_GREEN) return LIGHT_GREEN;
187     return LIGHT_OFF;
188 }
189
190 static void update_lights(void)
191 {
192     uint8_t light = blink_on ? mode_colour() : LIGHT_OFF;
193     setLightH(light);
194     setLightV(light);
195 }
196
197 void display_mode(void)
198 {
199     /* Mode 1: the traffic light runs (Exercise 10) */
200     if (mode == MODE_NORMAL) {
201         if (shown_mode != MODE_NORMAL) {
202             shown_mode = MODE_NORMAL;
203             traffic_init();          /* (re)start from S0 */
204         }
205         traffic_run();
```

```
206             return;
207         }
208 
209         /* Modes 2-4 (Exercise 6) */
210         if (mode != shown_mode) {
211             /* A new mode was entered: update everything once */
212             shown_mode = mode;
213             set7SegH(temp_value);        /* road H: the value  */
214             set7SegV(mode);              /* road V: the mode, e.g. 02 */
215             shown_value = temp_value;
216 
217             blink_on = 1;                /* start with the LEDs on */
218             setTimer(TIMER_BLINK, BLINK_MS);
219             update_lights();
220         }
221         else if (isTimerExpired(TIMER_BLINK)) {
222             /* Same mode: toggle the colour every 250 ms (2 Hz) */
223             blink_on = !blink_on;
224             setTimer(TIMER_BLINK, BLINK_MS);
225             update_lights();
226         }
227 
228         /* Exercise 7: the value was changed by B2 (or restored by B3) */
229         if (temp_value != shown_value) {
230             shown_value = temp_value;
231             set7SegH(temp_value);
232         }
233     }
```

Program 1.28: Sources in the top-level CMakeLists.txt (final)

```
1  target_sources(${CMAKE_PROJECT_NAME} PRIVATE
2      # Add user sources here
3      Core/Src/software_timer.c
4      Core/Src/input_reading.c
5      Core/Src/input_processing.c
6      Core/Src/led_display.c
7      Core/Src/traffic_light.c
8  )
```

1.10.3 Result

With the default durations, the traffic light in Proteus follows Table 9 of Exercise 1 exactly: one cycle lasts 10 s, and the red countdown of each road ends at the same moment as the amber light of the other road.

Table 29: Test results of Exercise 10

Test	**Result**
Reset (red 5, amber 2, green 3)	Mode 1 starts at S0: H red 05, V green 03; cycle 10 s
B1 in the middle of a cycle	Traffic light stops; Mode 2 edit display
Red 8, amber 3 (green adjusted to 5)	Mode 1 restarts at S0: H red 08, V green 05; cycle 16 s
Green 1 (red adjusted to 4)	H red 04, V green 01, then amber 3 s
B2 or B3 in Mode 1	No effect; the traffic light keeps running
Both roads, every moment	One road is red while the other is green or amber

All tests pass in Proteus. A software simulation with a mocked HAL, run for interrupt periods of 1, 10, and 100 ms, gives the same timelines and confirms over two minutes that the two roads are never green or amber at the same time.

References

[1] V. L. T. Trieu, *Lab 03 code + schematic*, GitHub repository. [Online]. Available: https://github.com/volethanhtrieu/Lab-exercise-MPU---MCU/tree/main/lab03.

[2] Ho Chi Minh City University of Technology, *Microcontroller Laboratory – Lab 03: Buttons/Switches*, course laboratory manual.

[3] STMicroelectronics, *STM32F103x4 and STM32F103x6 Low-density Performance Line Datasheet*, DocID15060, Rev. 7, June 2015. [Online]. Available: https://www.st.com/resource/en/datasheet/stm32f103c4.pdf. [Accessed: September 18, 2026].

[4] STMicroelectronics, *STM32CubeMX: STM32Cube Initialization Code Generator*. [Online]. Available: https://www.st.com/en/development-tools/stm32cubemx.html. [Accessed: September 18, 2026].

[5] STMicroelectronics, *STM32CubeF1 Embedded Software Package*. [Online]. Available: https://www.st.com/en/embedded-software/stm32cubef1.html. [Accessed: September 18, 2026].

[6] Labcenter Electronics, *Proteus Circuit Simulation Software with SPICE*. [Online]. Available: https://www.labcenter.com/simulation/. [Accessed: September 18, 2026].